

摘 要

本文简述了 USB 发展和研究的现状, 介绍并分析了 USB 系统软/硬件结构及特点。基于嵌入式系统、移动存储的发展趋势, 针对 USB Mass Storage (大规模存储) 类设备的数据传输, 提出一种嵌入式 USB 主机系统的设计方案。据此选择了 SPCE061A 微控制器及 SL811HS USB 主机接口器件, 构成 USB 主机系统的核心硬件结构。并外接液晶显示、键盘、扬声器等辅助硬件, 构建了一个既方便调试又具有一定实用性的硬件系统。在遵守 USB1.1 协议及 USB Mass Storage 类协议规范下, 软件实现采用分层的处理方法, 使软件主体从硬件相关过渡到硬件无关, 具有良好的可移植性。其中, 在分析了应用最为广泛的 FAT 文件系统基础上, 构建了一套精简的 FAT32 文件系统。使系统以文件形式传输数据, 并使其在嵌入式 USB 主机系统中传递的数据也能在 PC 机中识别, 增强了存储数据的兼容性。本文简要地介绍了该系统的调试/测试方法及结果, 并介绍了应用的例证。本设计方案突破了以往用 PC 机作为 USB 主机的局限, 实现了不依赖于任何操作系统, 利用微控制器控制 USB 移动存储设备传输文件的操作。扩展了 USB 系统的应用空间。通过与 USB Mass Storage 类设备进行数据传输的测试, 验证了该设计方案的可行性和优越性。提出的设计方案及测试流程对其它嵌入式 USB 主机的设计具有一定的借鉴意义。

关键词: USB USB 协议 嵌入式 主机 主机接口 Mass Storage 类

Abstract

This thesis firstly introduces the current researches on this issue and then the structure and characteristic of USB software and hardware system. Taking into account the latest development of embedded system and portable storage technique as well as the urgent demand of USB Mass Storage equipment for data transmission, the designer proposes a new design of embedded USB host system. With SPCE061A as micro controller chip and SL811HS as USB host interface device, the designer sets up a basic hardware framework. With other auxiliary hardware, such as LCD monitor, keyboard, and loudspeaker, she can adjust and test the system easily. Following the protocol of USB1.1 and the specification of USB Mass Storage protocol, the designer chooses to process the software into layers to ensure that different layers of software range from a hardware-involved model to a hardware free model, which makes the software more transplantable. By analyzing FAT file system, the designer works out a more simplified FAT32 file system, in which data is transmitted in files. Accordingly, the data transmitted in embedded USB host system could be identified in PC. Therefore, the compatibility of the data storage is ensured. The thesis briefly introduces the adjusting and testing method of the system as well as the test result, and analyzes a case of application. In stead of using PC as USB host, this design uses micro controller chip to control the data store equipment, which does not have to rely on any operating system; consequently, the application of USB system will be greatly enlarged. The test result of the

research justifies the feasibility and merits of this new design. The author believes that her method and testing flow could be valuable implications for later researchers.

Key words: USB USB protocol embedded Host Host interface
Mass Storage

第 1 章 绪论

1.1 引言

随着计算机技术的发展、嵌入式产品开发和应用的深入，品种繁多的移动设备层出不穷。移动数据存储和交换渐渐成为近年 IT 行业的热点。数据交换的可移动、方便快捷是推动数字化时代发展的基础。USB（通用串行总线）作为一种新型的接口技术从兴起不断走向成熟。它既是数据交换的媒介，又是其核心。

USB 总线采用的是星型结构，主从工作模式。USB 主机根据各个设备的属性，周期访问各个设备，USB 设备则是被动地响应 USB 主机的访问请求，所以 USB 的数据传输特点是所有 USB 设备都只能与主机建立连接。两台 USB 设备之间、两台主机之间无法直接相连进行数据传输。

怎样才能实现 USB 设备间点对点通信，降低成本，方便使用，使 USB 脱离 PC 机来进行相互数据的交换。这就要求一些设备能作为 USB 主机支持 USB 协议。在这种新的需求之下，嵌入式 USB 主机的研究成为 USB 领域新的关注焦点。USB 的协议栈和总线驱动的开发就变得必不可少，尤其是嵌入式设备的实时、小巧、方便携带等特性使得嵌入式 USB 主机的研究显得尤为重要。

1.2 USB 简述

1.2.1 USB 的发展

USB（Universal Serial Bus）是通用串行总线。1994 年 Intel，Microsoft 等七家公司共同推出了 USB 总线协议，它是一种新的外设连接

技术。以往传统的接口协议（串口、并口）一般只是针对其物理层做了定义和约束，而对于设备和 PC 间具体的通信行为和自身的拓扑结构则基本上没有涉及。随着计算机技术的飞速发展，这些接口已无法满足 PC 与外部设备之间不断提高的速度、稳定性、易用性等要求。USB 这一新型总线，就是为了解决 PC 机外围设备的拥挤和提高设备的传输速度而设计的。USB 以其高速度、低成本、低功耗、即插即用及使用经济方便等优点，迅速成为了 PC 机上的标准接口。而且成为了 PC 外部设备等与 PC 相连标准协议之一。USB 协议从 1.1 过渡到 2.0，作为其重要指标的设备传输速度也从 1.5Mbps 的低速和 12Mbps 的全速提高到了 480Mbps 的高速。扣除用于总线状态、控制和错误监测等的数据传输，USB 的传输速率仍远高于一般的串行总线接口。在工程中，USB 凭借它明显的优势，已大范围推广并使用到大规模数据存储系统中。USB 逐步走入了实用阶段、以惊人的速度发展。

无论是对于终端用户，还是对 USB 系统的设计者而言，USB 系统以简单、易用而著称。不同种类的 USB 设备使用相同的接口，为使用者和设计者提供了极大的方便。

USB 的发展见表 1-1，USB 的特点及应用范围见表 1-2。

表 1-1 USB 的发展

时间	版本
1994 年 11 月	USB 0.7
1994 年 12 月	USB 0.8
1995 年 4 月	USB 0.9
1995 年 8 月	USB 0.99
1995 年 11 月	USB 1.0FDR
1996 年 1 月	USB 1.0
1998 年 9 月	USB 1.1
1999 年 10 月	USB 2.0 (Draft 0.79)

1999 年 12 月	USB 2.0 (Draft 0.9)
2000 年 4 月	USB 2.0
2001 年 12 月	USB OTG (USB 2.0 的补充)

表 1-2 USB 的特点及应用范围

拓扑结构	星型 只有 host 一个主设备
最大设备数	通过 Hub 级联可以支持 127 个节点
连线特征	线长小于 5 米，2 根数据线，2 根电源线
传输方式	半双工、异步、串行、差分 支持控制、批量、中断、实时传输
热插拔	支持热插拔
传输速度	USB 1.X: 低速 1.5Mbps、全速 12Mbps USB 2.0: 高速 480Mbps
接口成本	低
应用方向	鼠标、键盘、猫、语音 (USB 1.X) 图像、高速存储 (USB 2.0)

1.2.2 USB 系统组成

USB 系统通过具有相同接口的串行连线将不同的设备连接到一起。USB 的物理连接是有层次的星型布局。每个集线器在星形的中心，每条线段是点点连接的，它通过 USB 集线器级联更多的 USB 设备组成 USB 系统。能够支持多达 127 个外设的同时连接，USB 支持音频设备、大容量存储设备 Mass Storage 以及人机接口设备 HID。USB 采用模块化，面向对象方法的结构。USB 系统组成如图 1-1 所示，由硬件和软件两大部分组成。

任何一个系统中，只有 1 个主机。USB 设备和主机系统的接口称作主机控制器。 主机控制器由硬件和软件结合来实现。根集线器是综合于主机系统内部的，用以提供 USB 的连接点。USB 的设备包括集线器和功能部件。USB 的协议实现了系统的协调。

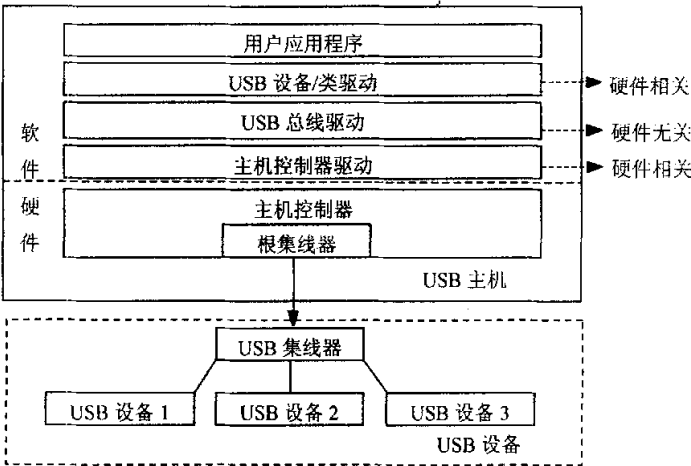


图 1-1 USB 系统组成

1.2.3 USB 的物理接口

USB通过一种4线电缆传送信号和电源。其中两根线D+和D-用于差分数据传输的数据信号线，另两根V_{BUS}和GND用于为USB设备提供电源。V_{BUS}的电压为+5V，为了保证足够的输入电压和终端阻抗，重要的终端设备应位于电缆的尾部，每个端口都可检测终端是否连接或分离，并区分出高速或低速设备。

1.2.4 USB 的电源

主要包括电源分配和电源管理两方面内容。电源分配指USB如何分配主

机所提供的能源。需要主机提供电源的设备称作总线供能设备。还有一些设备可自带电源，该类设备称作自供能设备。

1.2.5 总线协议

USB 是一种轮询方式的总线，主机控制器初始化所有的数据传送。每个总线执行动作按照传输前制定的原则。每次传输开始，主机控制器发送一个描述传输动作的种类、方向、USB 设备地址和端口号的 USB 数据包。这个数据包通常称为标志包（PID）。USB 设备从解码后的数据包中取出属于自己的数据。传输开始时，由标志包来标志数据的传输方向，然后发送端发送数据包，接收端发送一个握手的数据包表明传送是否成功。

1.2.6 USB 系统设置

USB 设备可随时安装或拆卸。所有的 USB 设备都是通过某个端口接在 USB 总线上。集线器有一个状态指示器，它可指明 USB 设备是安装还是拆除。在 USB 设备安装后，主机通过设备控制通道来激活该端口并以预设的地址值赋给 USB 设备。当 USB 设备从端口拆除后，集线器关闭该端口，并向主机报告该设备已不存在。对每个连接在总线上设备指定唯一的地址动作称为总线标号，因为 USB 允许设备在任何时刻安装或拆卸，所以总线标号是 USB 系统软件随时要作的动作。

1.2.7 USB 的数据流

USB 系统数据传输采用主机广播式请求，设备指定应答的方式。对于 USB 总线，所有连接在系统上的 USB 设备根据 USB 主机的访问请求分时占有数据总线，即所有 USB 设备共享系统带宽资源。

发送端和接收端之间的 USB 传输，可视为在主机和设备端口之间的一

条通道。总的来说，各通道之间的数据流动相互独立。

USB 协议规定了控制传输、批量传输、中断传输、同步传输 4 种基本的传输类型

1.2.8 USB 的主机

USB 的主机通过主机控制器与 USB 设备进行数据传输。USB 主机负责完成以下任务：

- (1) 检测 USB 设备的连接和断开。
- (2) 管理 USB 主机与 USB 外设之间的控制流。
- (3) 管理 USB 主机和 USB 设备之间的数据流。
- (4) 收集主机的状态和 USB 设备的动态信息。
- (5) 提供设备能源。

1.2.9 USB 的设备

通过总线与 USB 主机相连的称为 USB 设备，它们根据属性不同完成不同的功能，它们以从属的方式与 USB 主机进行通信，并受 USB 主机的控制。USB 设备分为 3 个层次：

- (1) 最底层提供了数据包收发的总线接口。
- (2) 中间层处理 USB 总线与数据收、发端之间的路由关系。
- (3) 最高层描述该设备的功能，如打印机接口等。

1.3 USB 主机工作原理

USB 主机是 USB 系统的核心。主机通过与设备相连的逻辑通道与设备进行通信；在通信的过程中，USB 主机充当源端或是目标端。USB 主机又可以分为 3 层：客户层、USB 系统层、主机控制器层。这 3 层分别与设备相

应的 3 层进行通信。图 1-2 给出了 USB 通信模型层次关系。

1.3.1 客户层

客户层处在USB主机的最高层，它是USB设备驱动程序的集合。客户与应用层之间进行逻辑的信息交流。为了编写输入/输出程序、屏蔽设备的硬件细节，USB引进了通道的概念。通道是客户与设备之间的逻辑连接。从设备输入数据到客户，即从通道取数据；从客户输出数据到设备，即往通道送数据。这样便实现了透明的通信。

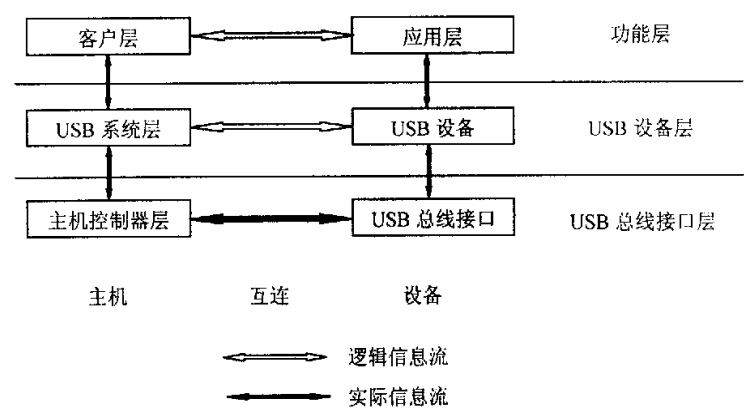


图 1-2 通信模型层次关系图

1.3.2 USB 系统层

USB 系统层是主机系统软件的一部分，是整个 USB 管理中枢。在主机的 3 个层次中，USB 系统层处于承上启下的地位。向上，它提供面向客户的接口；向下，它控制着主机控制器，使之完成 USB 所要求的工作。

USB 系统层又可以进一步划分为 3 个主要的组成部分：主机控制器驱

动（HCD）、USB 驱动程序（USB D）、主机软件。主机通信如图 1-3 所示：

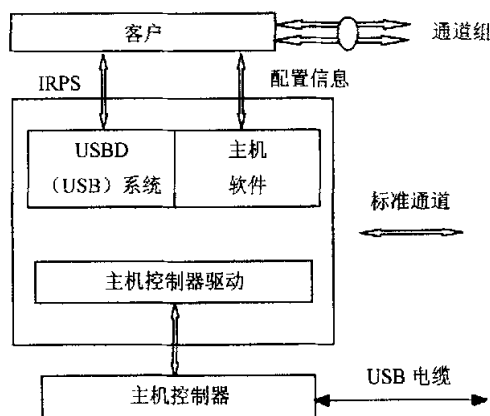


图 1-3 主机通信图

在 USB 系统中，USB D 处于 HCD 的上层。USB D 通过 HCD 所提供的 HCDI（HCD 接口）与 HCD 进行交互。同时，USB D 提供面向客户的接口，即 USB DI，以支持客户的数据传送、设备的配置等请求。主机软件是可选的，其方便了客户的操作。

HCD 与 USB D 提供不同层次的抽象。HCD 为上层的 USB D 提供了抽象的主机控制器，并将主机控制器与设备之间真正的数据通信过程加以抽象和简化。USB D 给上层的客户提供了 1 个抽象的设备，并将客户与设备之间数据通信过程加以抽象。USB 系统的 3 个部分协作配合，以完成 USB 系统功能。USB D 向客户提供了接口（USB DI），该接口体现了 USB 系统主要功能。USB D 与 HCD 之间的功能没有明确界线，USB D 与 HCD 结合起来，完成了如下功能：设备配置，资源管理。

1.3.3 主机控制器

主机控制器实际上是主机上总线接口。USB 主机与设备之间的通信最终都将通过 USB 电缆进行，即通过 USB 主机的总线接口和 USB 设备的总线接口之间相连的电缆进行。任何一个输出请求都是由主机控制器组织成包的形式发往总线。

USB 主机控制器位于 USB 主机层次结构的最底层，它完成了主机与设备之间的电气和协议的匹配，主要包括以下功能：串并转换，帧起始，数据处理，协议使用，主机系统接口等。

USB 系统层及主机控制器所提供的一系列功能，都是为了实现客户与设备之间透明的数据传输

USB 主机控制器提供了 USB 主机与 USB 设备之间连接的物理接口。图 1-4 显示了主机控制器的软硬件结构。

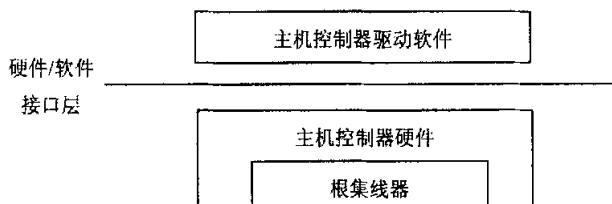


图 1-4 主机控制器的软硬件结构

1.4 嵌入式 USB 主机系统

传统的USB应用主要是为PC机与外部设备提供接口。随着USB应用领域的逐渐扩大，人们越来越希望USB能应用到各种没有PC机场合的领域中，尤其是在移动数据领域中。

嵌入式系统是广泛应用于IT产品中的微型计算机系统。是以应用为中

心，以计算机技术为基础，并且软硬件可裁剪，适用于应用系统对功能、可靠性、成本、体积、功耗有严格要求的专门计算机系统。它所使用的 CPU 一般为微控制器或微处理器，单片机系统是最为简单的嵌入式系统。

为了实现嵌入式系统使用 USB 设备的功能，根本办法就是构造一个以嵌入式系统为核心的 USB 主机功能模块。使其具有与 USB 设备进行数据传输的能力，形成嵌入式 USB 主机系统。

1.4.1 嵌入式 USB 主机的基本硬件结构

PC 机的处理器(CPU)基于 X86 结构，而嵌入式系统中的微处理器(MPU)或微控制器(MCU)种类繁多，不同的应用场合采用的 CPU 不同。从理论上讲，任何一种 CPU 都可成为控制 USB 主机系统的核心。对于这个系统，最关键的问题是 CPU 外扩总线能够满足 USB 总线的数据传输速度要求。为方便操作还需提供友好的人机界面，扩展键盘、通信等模块。

图 1-5 所示是嵌入式 USB 主机的基本硬件结构图。硬件部分的核心是 USB 主机接口芯片及 MCU 或 MPU。目前，比较成熟的 USB 接口芯片有 Cypress 公司的 SL811HS 和 Philips 公司的 ISP1161。由于 USB 协议框架的复杂性，选用 MCU 或 MPU 要求其具有足够的 RAM 来处理大量的数据缓冲区、大量的

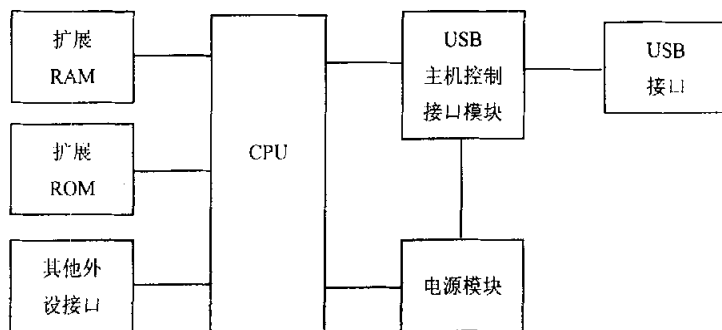


图 1-5 嵌入式 USB 主机基本硬件结构

变量及数据存储等。USB 定义了很多的类，用以支持不同类型的 USB 外设。不同的类有不同的驱动程序，因此要求系统有足够大的程序存储区来实现如 Mass Storage 类（即海量存储类）等。

1.4.2 嵌入式 USB 主机的软件

为了使设计的嵌入式 USB 主机系统具有通用性，整个系统的软件设计就不需要依靠任何操作系统。嵌入式 USB 主机，要实现完整的 USB 主机协议类，如设备的上电检测、分配地址、读取设备的各种描述符等基本操作。更重要的是要实现每一类 USB 设备的协议栈。不同类设备的协议栈都是一个特定功能的驱动程序。

（1）主机控制器驱动软件

通过数据线上的电平变化检测 USB 设备的接入和移出，实现复位设备、给设备分配地址、数据传输、设备的总线管理等功能。

（2）设备类协议软件

USB 协议规定了一系列设备类型。嵌入式系统中，没有足够的 RAM 和大容量的磁盘存储空间存储所有设备类协议。限于系统的性能和要求，只需要支持特定的设备类即可。对于一个应用 USB Flash 设备的 USB 主机，只要支持 USB Mass Storage 类。实现 USB Mass Storage 类所规定的各种命令。

（3）USB 用户软件

用户软件针对嵌入式 USB 主机系统支持的设备类而设计。USB Mass Storage 类设备支持 FAT 文件系统，以文件的形式管理和存储数据。因此，对于此类设备，就需要建立一个精简的文件系统。

1.5 论文的组织结构

第 1 章：介绍选题的背景、USB 系统组成、总线协议、数据流、阐述 USB 主机工作原理及嵌入式 USB 主机的结构体系。

第 2 章：主要介绍 USB 主机通信，USB 主机与设备的通信，USB 协议，USB 协议栈设备框架，USB 设备类。

第 3 章：介绍基于 SPCE061A 的嵌入式 USB 主机系统结构及硬件设计，各功能模块以及 USB 主机系统及各模块功能的硬件实现。

第 4 章：设计了嵌入式 USB 主机系统的软件，实现了嵌入式 USB 主机协议框架，进行了系统的分层处理，完成了 USB Mass Storage 类的软件实现。

第 5 章：从硬件层调试，系统软件层调试，给出了嵌入式 USB 主机 USB Mass Storage 类数据传输的调试方法和步骤。给出了读取优盘中语音文件的程序。对系统进行了分析。

结论部分对本文的研究工作进行总结。

第 2 章 USB 通信与 USB 协议

2.1 USB 通信

2.1.1 USB 通信结构

USB协议所说的通信指的就是USB设备和USB主机之间的通信。为了细化USB的通信机制，USB的开发者采用了分层的概念，每一层传输的数据结构对其他逻辑层是透明的。图2-1显示了USB设备与USB主机通信的逻辑结构和每层的逻辑通道。

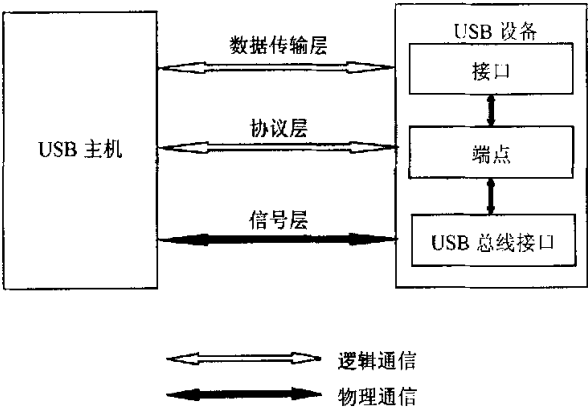


图 2-1 USB 通信的逻辑结构

信号层：实现在USB设备和主机的物理连接之间传输位信息流的信息。

协议层：实现在USB设备和主机端的协议软件之间传输包字节流的信息。

数据传输层：实现在USB主机端的客户端驱动程序和设备端的功能接口之间传输有一定意义的信息，这些信息在协议层被打包成包格式。

2.1.2 USB 主机通信

主机的通信就是应用软件与设备驱动程序之间以及驱动程序栈各驱动程序之间的通信过程。

USB 总线的驱动程序要实现对总线控制器的驱动、总线带宽管理、总线设备枚举等工作。USB 总线的驱动程序由 USB 主机控制器驱动，USB 协议栈，USB 设备驱动程序 3 部分构成。图 2-2 为 USB 驱动程序示意图。

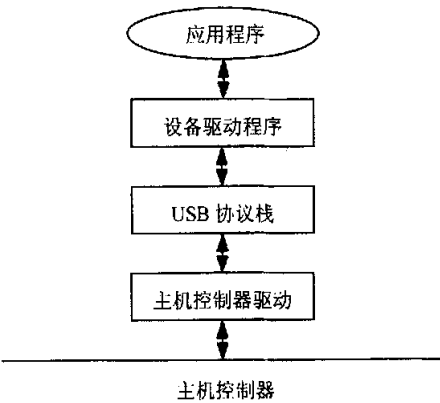


图 2-2 USB 驱动程序示意图

2.1.3 USB 主机与设备的通信

USB 的传输，是USB面向用户的、最高级的数据结构。协议逻辑上将设备分成了三层实体：总线接口、端点和功能接口，如图2-1所示。总线接口的功能除了传输和接收数据信号以外，逻辑上还包括识别设备的当前唯一

地址。设备端用于传输数据的接收点和发送点，有方向性（分为输入和输出）。功能接口或接口由一组端点组成，完成特定的功能。

主机与设备的通信也是建立在USB通信的逻辑结构上的。一方面用宏观和逻辑的角度，理解设备端点和管道的概念；另一方面用微观和物理的角度理解包和帧的概念。

1. 端点和管道

（1）端点

USB系统中每一个端点都有唯一的地址，由设备地址和端点号给出。因此，每一个USB设备对于主机而言就是一系列端点的集合。主机通过端点与设备进行通信。

端点0是默认的用作控制传输端点，其它端点必须在设备被主机配置后才能使用。

端点是一个USB设备中的唯一可寻址部分，是主机与设备之间通信流的起点和终点。是一个储存多个字节的缓冲区。每一个端点都有自己所支持的传输类型。

系统软件通过读取相应端点的端点描述符可以得到端点的传输类型。设备上的每个端点都有一个唯一的端点号，除了控制传输端点（端点0），其余每个端点只能是单向的（即输入或输出）。

（2）管道

主机与设备端点之间的连接称为管道，表示主机的数据缓冲区与设备端点之间交换数据的能力。设备被主机配置后端点即可使用，管道也就存在了。管道的概念主要用于PC机上驱动程序和用户程序的编写。

从逻辑上讲，USB数据的传输是通过管道进行的。USB系统软件通过缺省管道（与端点0相对应）管理设备，设备驱动程序通过其它的管道来管理设备的功能接口。

一个USB管道不是一个物理对象,它只是设备上的端点和主机软件之间的联系。

USB的数据传输在主机软件和一个USB设备的指定端口之间进行。通信协议首先要提出系统的通信模型,用以确定传输的数据、软硬件的实体及特殊功能。其次提供数据的传输方法,包括最底层的数据单元格式、硬件上的数据发送方式、整体的数据传输类型。再就是提供给用户的软件构架,如何在PC或嵌入式系统上通过软件编程来控制和使用这种通信协议。

实际的数据传输过程为:设备驱动程序通过对USB接口(USB Driver Interface)的调用发出输入输出请求包IRP;USB驱动程序接到请求后,调用主机控制器驱动程序接口HCDI(Host Controller Driver Interface),将IRP转化为USB的传输。一个IRP可以包含一个或多个USB传输;其后,HCD将USB传输分解为总线操作,主机控制器以包的形式发送给设备。

2. 包和帧

(1) 包

根据USB规范,USB总线上的数据总是以USB包的结构在总线上进行传输,而每个USB包都是由不同的“域”组成。为了满足不同类型的数据通信要求,USB总线支持不同类型的数据包结构。

“域”分为同步序列域、包标识域、地址域、端点域、帧号域、数据域和CRC校验域七类。

USB1.1支持的包类型包括:令牌包、数据包、握手包和特殊包。每一个包基本上包含了一个完整的信息。

令牌包:在USB系统中所有的事务处理都始于令牌包,它是由USB主机唯一地发送到总线上的。根据包标识域的不同,令牌包又分为起始令牌包(SOF)、输入令牌包(IN)、输出令牌包(OUT)和用于控制传输的设置令牌包(SETUP)。

数据包:数据包中的包标识域类型为数据类型,数据包中装有主机和

设备之间传输的数据信息。有两种类型，靠包的标识符DATA0和DATA1来区分。

握手包：握手包用来报告事务处理过程中接收方的状态。

(2) 帧

从时间单位上来看，一帧的长度为1ms，系统中所有的事务处理过程都是在一个个以帧为单位的时间周期内完成的。一帧可以容纳四种传输类型的许多事务处理过程，USB主机决定某段时间内和哪个设备完成一次事务处理过程。

3. 事务和传输

(1) 事务

每一次数据交换的过程叫做“事务”，包是USB事务的基本构成单位。每一个传输包括一个或多个事务。每种传输方式都由很多个事务来完成。一个事务可以在一个帧里完成，也可以使用多个帧。

各种事务有不同的格式，事务格式根据端点类型而变化，可分为批处理、控制、中断及同步4类。

每个事务一般分为三个阶段。第一阶段发送令牌包。每个事务总由主机发送一个令牌包开始，它定义了事务的类型以及设备和端点的地址。共有3种令牌包，IN：从USB设备读数据，OUT：向USB设备发数据，SETUP：在枚举过程中使用。表示将要开始一个控制传输。

第二阶段为数据包。令牌包后面是可选的数据包，DATA0和DATA1。使用两种数据包是为了防止数据发送方和接收方比特序列失调。该阶段负责运送和传输相关数据。

第三阶段发送一个握手包。是主机或设备用来向对方报告一个命令或数据处理操作的状态。对数据发送方提供一个反馈信号，通知发送方数据是否已经被正确接收到了。握手包有3种。ACK：成功接收到包。NAK：表示

设备忙无法接收或发送数据。只能由设备方给出。STALL：说明设备端点因为严重的错误而被禁止，也只能由设备方给出。

(2) 传输

传输是由若干个事务组成的。针对USB1.1而言, 以USB包为基础定义了控制传输、批量数据传输、中断数据传输、同步数据传输等四种基本的数据传输类型。

控制传输: 用来传输命令和状态。一般在枚举过程中使用控制传输来识别和配置该设备。

中断传输: 用于支持数据量少的周期性传输要求, 主要用于键盘、鼠标等设备上。

批量传输: 用于支持突发的大量的数据, 对连续性不作要求, 它可以利用任何可获得的带宽。通常用于打印机、移动硬盘。

图2-4 显示了从客户软件到总线的USB信息转换。

等时传输: 用来传输量大、实时性强但准确性要求不高的数据。通常用于音视频设备。

2.2 USB 协议

2.2.1 USB 协议栈设备框架

1. 标准的USB描述符

USB协议为USB设备定义了一套描述设备功能和属性的有固定结构的描述符。当USB设备连接到主机时, 主机要对该设备进行配置, 配置之前主机要知道这是哪一类设备, 具有哪些功能, 使用什么传输方式, 以及所要传输的数据量大小。主机确认这些信息后, 才能进行配置。这些信息是存储在设备中的由USB描述符中体现出来的。描述符是一个完整的数据结构, 通

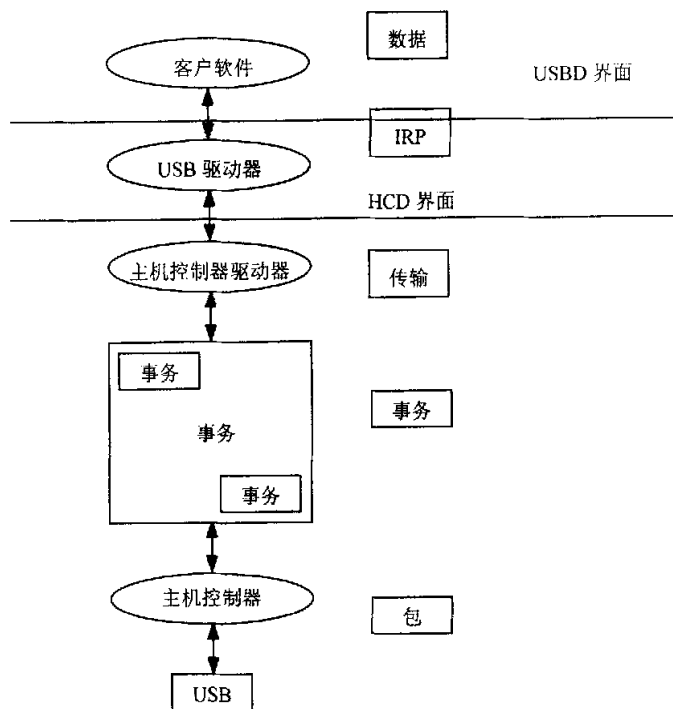


图 2-4 从客户软件到总线的 USB 信息转换

过编程描述一个USB的所有属性，并存储在USB设备中。主机要通过一些标准的请求命令来询问设备，设备才能把这些描述符发给主机。USB设备的属性包括很多内容，USB协议将这些信息做了分类，定义了很多描述符，一些标准的描述符，包括：

- 1- 设备描述符
- 2- 配置描述符
- 3- 字符串描述符
- 4- 接口描述符
- 5- 端点描述符

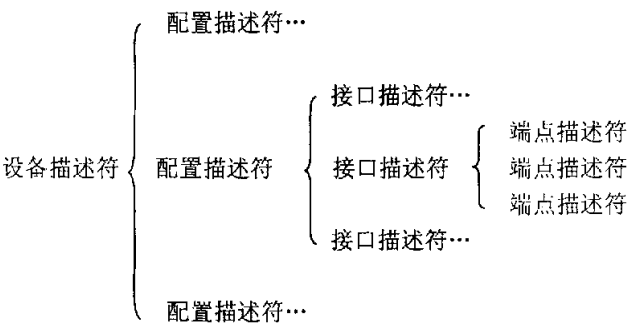


图 2-2 标准 USB 描述符结构图

这些描述符之间具有一定的关系,如图 2-2 所示。设备描述符的级别最高,端点描述符的级别最低。

2. USB 标准命令

控制传输是最重要也是最复杂的一种传输类型。控制传输的“初始设置步骤”中,在数据包阶段,包含了 DATA0 的数据包。这个数据包是一个 8 字节的标准请求命令,是主机用来发送控制阶段的请求命令的,这些请求命令是主机配置设备的关键。

所有的 USB 设备请求命令都是主机通过缺省的控制管道利用控制传输来发送的。这些命令具有标准的格式,共 11 个,如表 2-1 所示。

表 2-1 11 个标准命令的编码

请求命令名称	编号 (bRequest 的值)
获取状态请求 GET_STATUS	0
消除特性请求 CLEAR_FEATURE	1
设置特性请求 SET_FEATURE	3
设置地址请求 SET_ADDRESS	5

获取描述符请求 GET_DESCRIPTOR	6
设置描述符请求 SET_DESCRIPTOR	7
获取配置请求 GET_CONFIGURATION	8
设置配置请求 SET_CONFIGURATION	9
获取接口请求 GET_INTERFACE	10
设置接口请求 SET_INTERFACE	11
同步帧请求 SYNC_FRAME	12

这些标准的请求命令中，用的最多的还是 GET_DESCRIPTOR、SET_ADDRESS、SET_CONFIGURATION。设备插入后，首先要在缺省地址（地址 0）下取得设备描述符，在描述符中得到很多有用的信息，再利用这些信息来对设备进行配置。在配置之前要先给设备安排地址。所以这几个命令是最常用的也是必须要使用的。

2.2.2 USB 设备类

把常用的计算机外设根据其功能特性分类，并为每类设备制定了设备与主机进行数据传输的标准。这样就使得具有相同属性的不同设备共用同一设备驱动程序成为可能。

在USB设备类规范中，规定了不同属性USB设备的设备配置和数据传输特性。

设备类是个庞大的概念。针对USB的每一种设备类都有一套特殊的设备类协议。不同速度、不同大小的USB设备，被USB采用设备类的方式进行分类。使得USB总线能够有效地控制和管理各种设备，并使各种设备的开发变得规范、简便。

类，既是一组在属性和功能上具有相同之处的设备或接口的集合。

按照USB通用设备类协议的定义，USB的类及其相关的设备驱动程序起2

种作用：

其一：设备的接口与主机之间的通信方式。包括数据的发送机制和控制方式。

其二：接口提供的功能。即规定了完整的或部分的接口所提供的功能USB以设备类协议的形式对每种类进行定义。

到目前为止，USB定义的各种标准设备类，有基于设备定义类的，有基于接口定义类的，也有既基于设备也基于接口的。诸如：

语音设备类：针对麦克风、音箱等语音/音频设备

通信设备控制类：针对电话、调制解调器等设备

影像设备：针对扫描仪、数码相机等设备

人机交互设备类：针对键盘、鼠标、游戏杆等设备

存储设备类：针对CD-ROM、磁带、软盘机等设备

电源设备类：针对电源控制等设备

打印机设备类：针对打印机设备

监视器设备类：针对显示器等设备

具体的每一种类协议都有各自的说明。

2.2.3 Mass Storage 类概述

Mass Storage 类（即海量存储类）主要用于为软磁盘接口、ATA 接口及 Flash 存储器等设备建立的 USB 接口，这一类的特点是数据交换量大，有可能直接涉及文件的各种操作，并且支持不同的数据存储载体接口本身的一些操作命令。不同的数据存储载体接口就构成了 Mass Storage 类的子类。

各种子类都有一套自己的协议，并通过命令的形式实现各种功能。这些命令都是各子类特有的，而与 USB 没有任何关系。在 USB 进行数据传输的时候，这些命令都被打包成一系列的命令包进行传输。而真正与 Mass

Storage 的传输方式相关的是其传输协议。有关 Mass Storage 类的标准描述符等也将在其传输协议中实现。

第3章 嵌入式USB主机系统结构及硬件设计

3.1 嵌入式USB主机系统结构

嵌入式USB主机硬件系统的核心在于USB主机接口模块和主控制模块。其中，主控制模块是系统的核心控制部分，由以SPCE061A为主的嵌入式系统组成。根据本系统的设计要求设计的系统原理框图如图3-1所示。

系统分为4个主要部分：其中，主机部分由主控制模块、主机接口模块和电源模块组成。为了系统调试方便扩展了液晶显示模块。还有1个用PROBE模块实现的在线调试模块，通过在线调试器与PC机通信，另外还有状态指示、按键和扬声器。

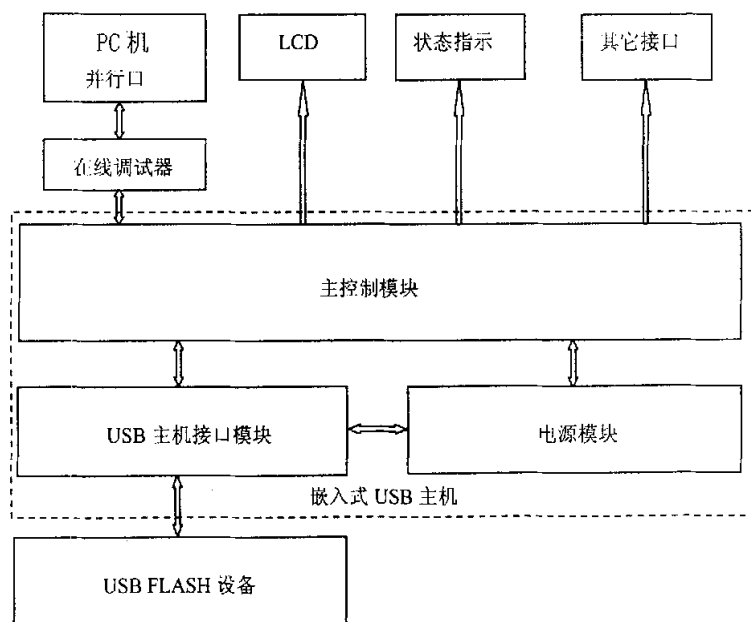


图 3-1 基于 SPCE061A 的嵌入式 USB 主机系统框图

3.2 主控制模块

主控制模块是系统的核心控制部分用 SPCE061A 实现。作为整个 USB 协议框架是非常复杂的。所使用的 MCU 或 MPU 应具有足够的 RAM 用以处理大量的数据缓冲区、大量的变量、数据存储等。同时,系统要有大量的程序存储区,来满足 USB 定义的很多类的 USB 外设。对于我们要实现的 Mass Storage 类就需要 10K 的 Flash 空间。若要实现播放优盘内存储的信息,就考虑到 SPCE061A 强大的语音功能,所以选用 SPCE061A 作为本系统的控制器。

SPCE061A 是一款由台湾凌阳公司推出的 16bit 的 $\mu^{\prime}$ Nsp[™]微处理器。是比较新的一款单片机。它在语音处理方面非常有特色。已经在语音识别、非特定人语音命令识别、特定人语音命令识别、带语音播报的测量仪器,以及其他领域都有设计和应用。本文研究并实现了基于 SPCE061A 的嵌入式 USB 主机的功能。为大专院校的教学、电子竞技、应用系统的设计与开发提供了新的应用领域。本文所设计的嵌入式 USB 主机利用了 SPCE061A 独特的语音处理功能,能够与 USB 设备进行数据交换,通过 SPCE061A 本身来播放优盘中的语音文件。同时,设计时考虑可移植性,采用分层设计以适用于不同的 MCU。

SPCE061A 是为适应控制处理、数据处理以及数字信号处理等领域的发展而设计的。其结构如图 3-2 所示。其 CPU 内核采用凌阳最新的 $\mu^{\prime}$ Nsp[™](Microcontroller and Signal Processor)16 位微处理器芯片。采用的是模块式集成结构,以 $\mu^{\prime}$ Nsp[™]内核为中心集成不同规模的 ROM、RAM 和各种功能丰富的外设接口部件。

SPCE061A 在 2.6V~3.6V 工作电压范围内的工作速度范围为 0.32MHz~49.152MHz。

内置 2K 字 SRAM 和 32K 字 FLASH 仅占一页存储空间。

32 位通用可编程的多功能 I/O 端口。

两个 16 位可编程定时器/计数器。

具有低电压复位 (LVR) 功能/低电压检测 (LVD) 功能。

7 通道 10 位模/数转换器 (ADC) 和单通道声音模/数转换器。

声音模/数转换器输入通道内置麦克风放大器, 并具有自动增益控制 (AGC) 功能。

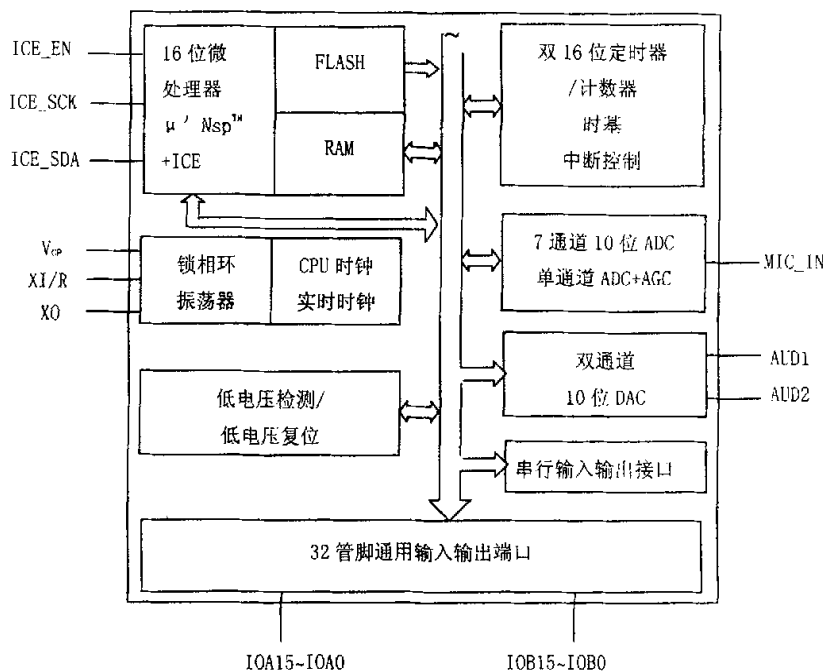


图 3-2 SPCE061A 的结构

两个 10 位 DAC 方式的音频输出功能; 使用凌阳音频编码 SACM-24 方式 (2.4kb/s), 能容纳 210 秒的语音数据。

可编程音频处理。

14 个中断源来自定时器 A/B、时基、2 个外部时钟源输入和键唤醒。

指令系统提供具有较高运算速度的16位×16位的乘法运算指令和内积运算指令,为其应用增添了DSP功能……。

较高的处理速度使SPCE061A能够非常容易地、快速地处理复杂的数字信号。开发数字声音和语音识别产品,选择SPCE061A是一种最经济的选择。

本文所设计的嵌入式USB主机,它主要是实现SPCE061A与优盘之间的数据交换。系统具有三个功能:1.检测USB外设是否连接成功;2.对优盘进行存取文件的操作;3.播放存储在优盘中的语音文件。

3.3 USB主机接口模块

USB主机接口模块实现USB主机的接口,配置下行USB设备,管理USB总线电源。

3.3.1 SL811HS 芯片

SL811HS的功能模块框图如图3-3所示。SL811HS是Cypress公司针对嵌入式系统USB应用而设计的接口器件。支持USB1.1的全速和低速设备。提供USB主机的硬件接口及总线管理的物理机制。主机控制器实现了标准的USB主机串行接口引擎(SIE)、根集线器功能,通过数据/地址总线与SPCE061A相连。因此,满足了嵌入式USB主机系统所需要的功能。即由主机接口芯片来完成总线底层物理数据包的发送、接收,并能检测总线状态,发出中断请求。实现了USB总线接口层的功能。其主/从控制器正是嵌入式USB主机结构中的嵌入式USB主机控制器部分。并且SL811HS还可以作为USB设备的控制器。

SIE用来实现数据并、串转换。

处理器接口提供了数据、地址和控制线,使得SL811HS能够和控制器

进行通信，并受 CPU 的控制。

中断控制器通过检测各种 USB 总线的状态变化，以中断信号的方式报告给外部的核心 CPU。

RAM 一方面提供了存储数据的缓冲区，另一方面又定义了各种寄存器用于控制 SL811HS，并对 USB 的各种状态进行检测。

时钟发生器模块需要外接晶振来为整个 SL811HS 提供时钟。

集成的根 Hub 提供 USB 设备的连接点。

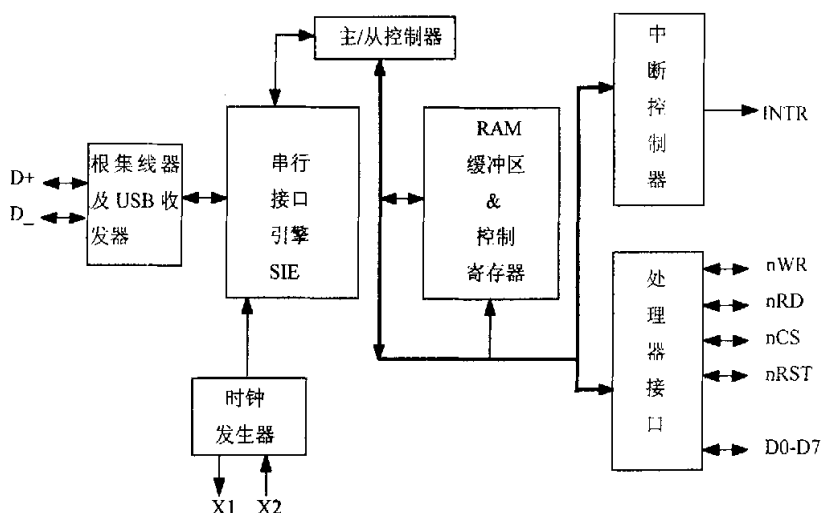


图 3-3 SL811HS 结构示意图

SL811HS 的主要特点是：

- 1) 能通过硬件或软件设置的方法使芯片工作在主机或者设备的模式。
- 2) 遵循 USB 协议 1.1。
- 3) 自动探测所接设备是全速设备还是低速设备。
- 4) 8 bit 的双向数据总线，片上集成了 SIE、单端口根 Hub、USB

收发器。

5) 3.3V 工作电压，接口兼容 5V 电平。

6) 硬件自动产生 SOF 帧起始包、自动生成令牌包、数据包中所需的 CRC5/16 校验数据。

7) 内部 256Byte 的 RAM，支持乒乓数据传输功能。

SL811HS 仅通过一根管脚来区分地址和数据传输，并支持地址自增模式。USB 主机接口电路连接如图 3-4 所示。

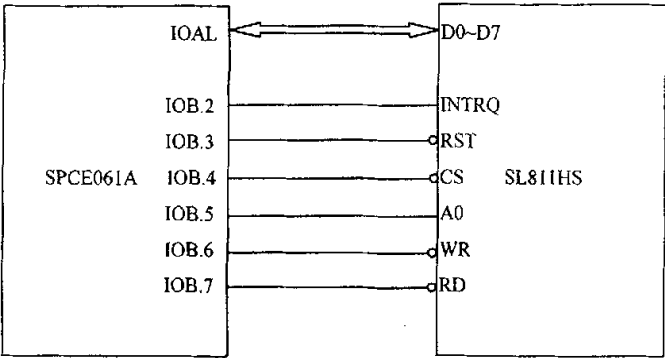


图 3-4 SPCE061A 与 SL811HS 的接口

这些总线信号包括数据信号线、读/写控制信号线和片选信号线、SL811HS 的中断请求信号线。SPCE061A 的通信引脚分配如表 3-1 所示。

表 3-1 SPCE061A 的通信引脚分配

信号说明	SPCE061A 的信号引脚	输入/输出	SL811HS 对应的引脚	输入/输出
数据信号	IOAL. 0~7	输入/输出	D ₇ ~D ₀	输入/输出
读信号	IOBL. 7	输出	nRD	输入
写信号	IOBL. 6	输出	nWR	输入

片选信号	IOBL. 4	输出	nCS	输入
SL811HS 复位信号	IOBL. 3	输出	nRST	输入
地址/数据选择信号	IOBL. 5	输出	A0	输入
SL811HS 中断请求信号	IOBL. 2	输入	INTRQ	输出

由于系统实现 USB 主机的功能，所以将 SL811HS 的 M/S 管脚（主机或设备工作方式）直接接地，让 USB 接口工作在主机方式。SPCE061A 的 IOBL. 5 与 SL811HS 的 A0（模拟输出）相连，表明系统以 IOBL 的第 6 位来区分访问此 USB 接口，其中 SL811HS 的 A0 功能较为灵活，它为低电平表示数据线上的数据是 SL811HS 的 RAM 空间的地址指针；而置位 A0 实现读写指定地址的 RAM 数据，从而实现数据的传输。而且它支持地址自增模式，即可将连续地址单元中的内容读出传输，而不用再次写入地址指针。从而大大提升数据传输的有效率。系统采用查询的方式来识别 USB 设备是否插上，当有设备插到接口上时，进入 USB 设备驱动。

3.3.2 SL811HS 外围电路设计

本系统采用 28 引脚 PLCC 封装形式的 SL811HS 芯片。

1. 芯片供电

SL811HS 采用 3.3V 电源供电，其 I/O 端口驱动电压也是 3.3V，但是可以兼容 5V 逻辑，所以芯片 3 个 V_{DD} 都接 +3.3V。

2. 时钟电路

本系统中为了提高电路的稳定性，SL811HS 不采用 12MHZ 晶振，时

钟模式引脚接地，直接采用 48MHz 的有源晶振作为时钟源。

3. USB 下行端口设计

本嵌入式 USB 主机系统要实现读取优盘数据的功能，主机系统连接的外设仅为耗电量极低的 USB 闪存盘。设备采用总线供电。USB 接口由电源线 (V_{cc})，地线 (GND)，差分数据线 (D+, D-) 共四根线组成。所以给 USB 下行端口的 V_{cc} 接 +5V 电压。在主机方 USB 接口的数据线 D+、D- 上接有 15K 的下拉电阻，当 USB 设备未连接到主机的端口时，由于下拉电阻的作用使数据线接地。在设备方 USB 接口的数据线上接有 1.5K 的上拉电阻。USB1.1 协议规定低速设备在 D- 连接上拉电阻，全速设备在 D+ 连接上拉电阻。当一个设备连接到主机，设备端数据线上的 1.5K 上拉电阻与主机端 15K 的下拉电阻就会形成一个分压器。因此，其中一条数据信号线 (D+ 或 D-) 的电位将被提升到电源电压的 90% 左右。此时，主机通过检测哪根数据线发生了变化即可检测出是低速还是高速设备设备已经连接上。

4. 复位电路：

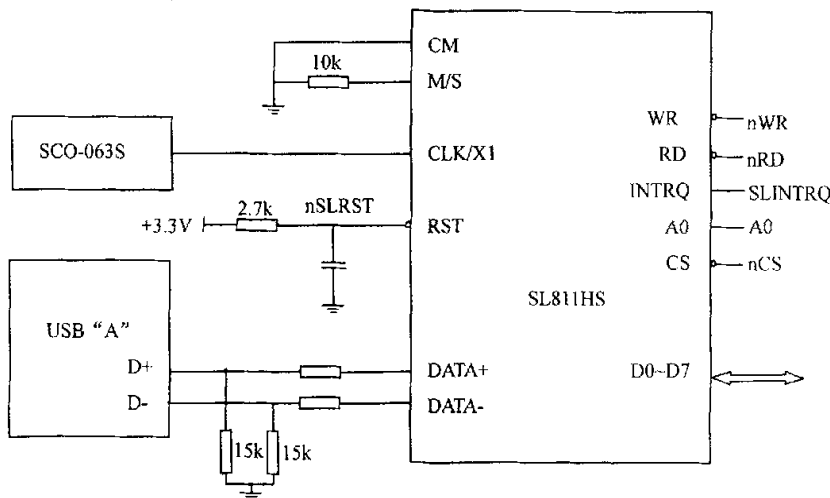


图 3-5 SL811HS 外围电路

采用如图 3-5 所示 SL811HS 外围电路中的复位电路部分，系统在上电时就能复位，也可以通过 SPCE061A 来控制，实现软件复位。

3.4 显示模块

显示模块用于显示 USB 主机系统的信息。

3.4.1 液晶 RT12864 电路

本系统采用的液晶显示模块是RT12864 汉字图形型液晶显示模块，可显示汉字及图形，内置8192 个中文汉字（16×16 点阵）、128个字符（8×16 点阵）及64×256点阵显示RAM，GDRAM可与CPU直接接口，提供两种界面来连接微处理器，8-位并行及串行两种连接方式，具有多功能光标显示，画面移动，睡眠模式等。显示汉字时最多可以显示4行，每行8个汉字，每个汉字对应液晶上一个地址，当显示字符时，每个地址对应位置可以显示两个字符，其地址对应关系如表3-2所示。液晶RT12864引脚定义如表3-3所示。

表3-2 液晶RT12864显示字符与地址的对应关系

Line1	80H	81H	82H	83H	84H	85H	86H	87H
Line2	90H	91H	92H	93H	94H	95H	96H	97H
Line3	88H	89H	8AH	8BH	8CH	8D	8EH	8FH
Line4	98H	99H	9AH	9BH	9CH	9DH	9EH	9FH

表3-3 液晶RT12864引脚定义

引脚号	引脚名称	功能说明
1	VSS	模块的电源地
2	VDD	模块的电源正端

3	VO	LCD 驱动电压输入端（悬空）
4	RS（CS）	并行的指令/数据选择信号 0：指令；1：数据，串行的片选信号
5	R/W（SID）	并行的读写选择信号 0：写；1：读，串行的数据口
6	E（SCLK）	并行的使能信号（读写时用），串行的同步时钟
7	DB0	数据 0
8	DB1	数据 1
9	DB2	数据 2
10	DB3	数据 3
11	DB4	数据 4
12	DB5	数据 5
13	DB6	数据 6
14	DB7	数据 7
15	PSB	并/串行接口选择：H-并行；L-串行
16	NC	空脚
17	RET	复位信号，低电平有效
18	NC	空脚
19	LEDA	背光源正极 LED+5V
20	LEDK	背光源负极 LED-0V

3.4.2 液晶 RT12864 与 SPCE061A 的接口设计

根据液晶 RT12864 引脚定义，设计如图 3-6 所示的液晶 RT12864 与 SPCE061A 的接口电路。

3.5 电源模块

USB 总线需要 5V 电压供电，液晶 RT12864 可以工作在 5V 电压模式下

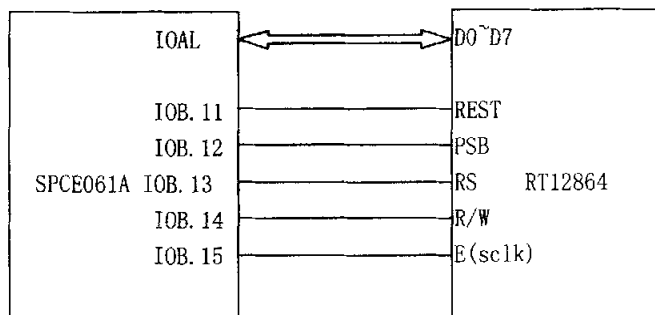


图 3-6 SPCE061A 与 RT12864 的接口

也可以工作在 3.3V 电压模式下，但是液晶驱动电压需要 +5V，LED 背光也需要 5V 电压，SL811HS 工作电压为 3.3V。所以需要设计出一个合理的供电电路，具体方案如下：

系统直接输入 5V 电源，采用 AS1117 作为电压转换芯片，将 5V 的输入 V_{cc} 转换为 3.3V，可供 SL811HS 系统使用。为此设计了一个跳线端子，适合多种供电方式下的情况，如图 3-7 所示。

端子“AS1117 输出”指电源变换电路中芯片 AS1117 的输出，输出电压是 +3.3V， V_{cc} 是来自 SPCE061A 模块电路的系统电源，可以是 +5V，也可以是 +3.3V。端子“+3.3V”、“+5V”是给接口板提供的电源电压。具体从哪个端子上引要看 SPCE061A 模块电路提供的电压是多少。

1) CE061A 模块输出 +5V 时，则需要一个变换电路产生 +3.3V 电压给 SL811HS 供电，跳线器将 3 和 4 短路，这样 +5V 端就来自 SPCE061A 模块电路。此时再将 1 和 2 短路，+3.3V 来自 AS1117，从 +5V 变换后得到。

2) 当 SPCE061A 模块电路输出 +3.3V 时，则接口板还需要外接一路 +5V 电源，此时只需要将跳线器短路 2 和 3，+3.3V 来自 SPCE061A 模块电路，+5V 来自接口板上的外接电源。

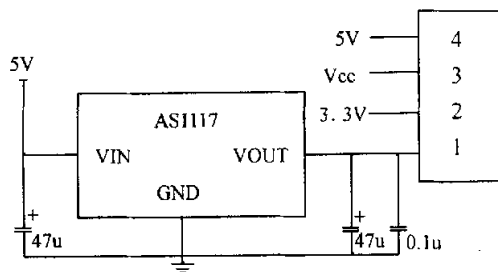


图 3-7 电压转换电路

鉴于以上各功能模块的分析和设计，基于 SPCE061A 的嵌入式 USB 主机电路原理图如图 3-9 所示。

基于 SPCE061A 的嵌入式 USB 主机的硬件电路如图 3-8 所示。

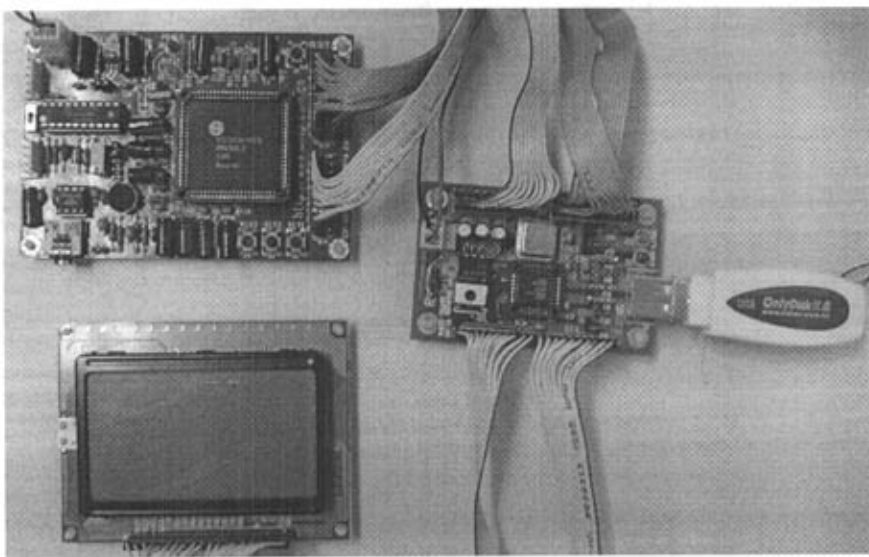


图 3-8 基于 SPCE061A 的嵌入式 USB 主机的硬件电路

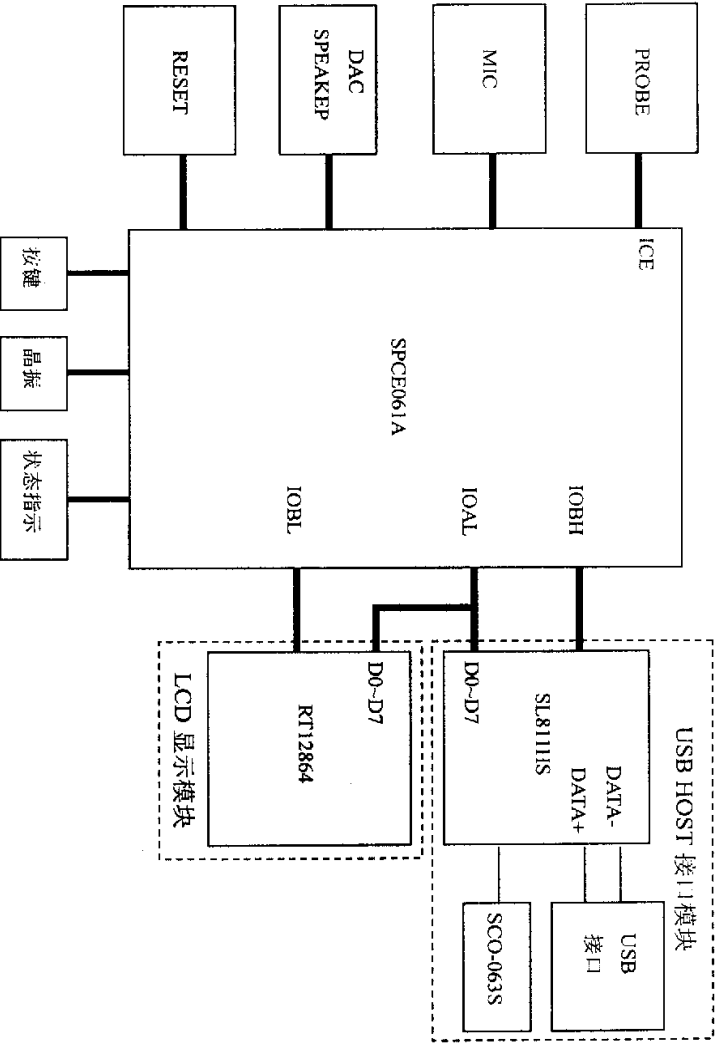


图 3-9 基于 SPCE061A 的嵌入式 USB 主机电路原理图

第4章 嵌入式USB主机系统的软件设计

USB 主机软件部分由 USB 主机控制器驱动程序 (HCD), USB 驱动程序 (USB D) 和 USB 设备驱动程序 3 个模块组成。其中 USB D 是整个 USB 主机软件的核心部件。USB D 一般负责管理所有的 HCD、设备驱动程序和所有连接在 USB 总线上的设备, USB D 通过接口函数将 USB 数据包发送到 HCD 层, 并通过 HCD 驱动将数据发送给相关的 USB 设备。HCD 和 USB D 在 USB 规范中被称为主机端系统软件或协议软件。USB 设备驱动程序及根集线器称为客户端驱动程序。客户端驱动程序不直接与 USB 设备硬件打交道, 而是通过协议软件的抽象处理实现与设备的不同功能接口之间的通信。HCD 主要完成主机控制器硬件的初始化, USB 链路级数据传输, 根集线器控制等。USB 主机软件结构如图 4-1 所示。根据系统实际应用需要, 在软件编程中按照实际的软件分层方式实现了嵌入式 USB 主机软件。减小了与硬件相关的软件量, 提高软件系统的可移植性。

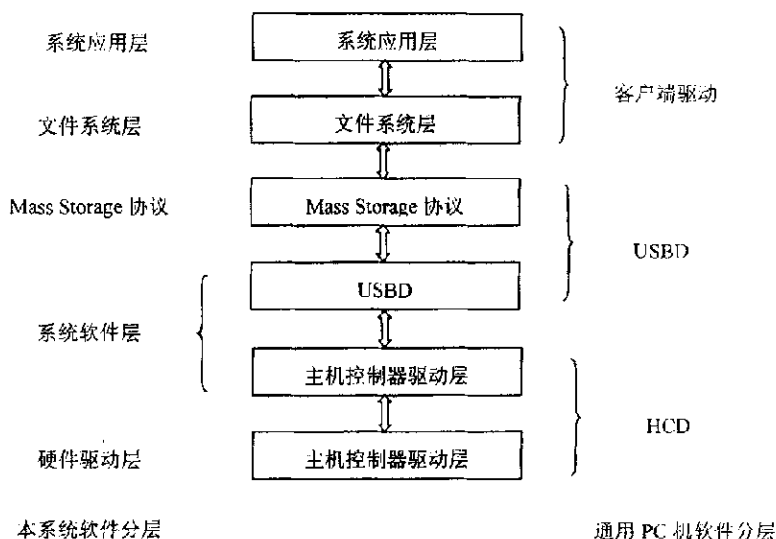


图 4-1 USB 主机软件结构

图 4-1 USB 主机软件结构

4.1 USB 主机协议框架的软件实现

USB 国际组织定义了大容量存储类设备 (Mass Storage Class) 的规范，包括 4 个独立的子类规范：USB Mass Storage Class Control/Bulk/Interrupt(CBI)Transport; USB Mass Storage Class Bulk-Only Transport; USB Mass Storage Class ATA Command Block; USB Mass Storage Class UFI Command Specification。前两个子规范定义了数据/命令/状态在 USB 上的传输方法。Bulk-Only 传输规范仅使用批量 (Bulk) 端点传输数据/命令/状态。后两个规范定义了对存储介质的操作命令。UFI 命令规范是针对 USB 移动存储类设备而制定的。UFI 命令规范是基于 SCSI-2 命令规范发展起来的。Bulk-Only 传输规范是系统通过默认管道进行枚举后，仅使用批量传输数据/命令/状态。其中有三种类型的数据在 USB 主机和设备之间传送。即 CBW、CSW 和普通数据。

USB 大容量存储类主机设计的难点是将 USB 主机驱动、数据通信传输和文件系统高效能的匹配，实现 USB 数据流的高速、无误传输。本系统实现的是读写 U 盘数据的功能。作为嵌入式系统，针对我们所要实现的特定功能将整个系统的软件分为四个层次：硬件驱动层、系统软件层、Mass Storage 层、系统应用层。这四层的结构层次如图 4-2 所示：

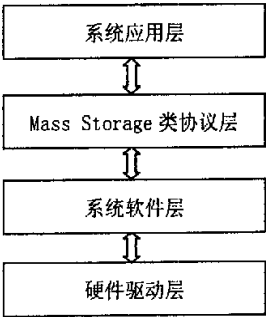


图 4-2 嵌入式 USB 主机系统软件层次图

4.2 硬件驱动层

硬件驱动层完成最底层的功能，直接和接口芯片打交道，实现 SPCE061A 与 SL811HS 的通信以及完成对 RT12864 的寄存器、缓冲器的读写。SPCE061A 与 SL811HS 的通信是整个 USB 体系软件的根基，包括了 SPCE061A 对 SL811HS 的复位、数据的读写、设定地址等函数。主要包括了以下几个函数：

```
void SL811Write( int Address,int Date ); 写 SL811HS 寄存器  
void SL811Read( int Address ); 读 SL811HS 寄存器  
void SL811BufWrite( int DestAddress,int SourcAddress,int  
Length ); 写 SL811HS 缓冲区  
void SL811BufRead( int SourcAddress,int DestAddress,int  
Length )。读 SL811HS 缓冲区
```

读写 SL811HS 缓冲区函数的第一个参数都是指 SL811HS 的地址，第二个参数都是指 CPU 的内存地址，第三个参数为要传输的数据的长度。

若有大量的数据需要传递就考虑到 Ping-Pong 传送。通过 SL811HS 向设备发送数据的 Ping-Pong 方式具体过程为：

```
向 SL811HS 的数据缓冲区中同时写入两个帧  
发送第一个帧  
等待确认信息  
收到确认信息后发送第二个帧，在第一个帧的位置上填充第三个帧，  
同时开始等待确认信息。
```

对于液晶来说，由于寄存器和缓冲器不是统一编址，由专门的控制信号先来控制是访问命令寄存器还是访问数据缓冲器，所以相对应一共有三个函数：

```
void Write_Command( int Command );
```

```
void Write_Data( int Data );  
int Read_Data( );
```

4.3 系统软件层

USB 系统软件根据功能分成 USBD 和 HCD 两部分。HCD 为上层提供了主机控制器的抽象以及数据在总线上传输的抽象。主机驱动层将建立/输入/输出请求转换为事务或将事务转换为请求,并对它们进行组织,从而调控 USB 主机控制器。USB D 为上层的客户端驱动程序提供了 USB 设备的抽象,并在客户端驱动程序和所驱动的设备之间提供了数据传输的抽象。USB 驱动层通过对主机驱动层的调用和响应调用,转换从设备端点来或到设备端点去的请求中的数据,如一个请求传输可能需要多个传输才能完成。在系统设计时将这两层驱动和二为一,成为 USB 主机最底层的设备驱动。

USB 系统软件的功能主要包括:配置设备;创建管道、分配带宽;数据传输。实现基本的 USB 协议,完成设备的检测、设备的枚举(控制传输)和数据的传输(批量传输),其中设备枚举中又包括总线复位、设置地址、设置配置、设置接口等操作,每一个设置都是一个控制传输。

1. 嵌入式 USB 数据传输中的事务

USB 中的事务处理是 USB 总线上数据传输的基本单位,有了事务才能进行顶层的控制传输和批量传输。主要是通过函数 USBTransaction()来实现。Transaction 流程图如图 4-3 所示。

2. 嵌入式 USB 主机中数据传输

对于本嵌入式 USB 大容量存储类主机数据通信传输是建立在底层通信传输之上的。完成两种类型的传输:控制传输和批量传输。在设备刚插入时要使用控制传输方式对设备进行枚举,之后采用批量传输方式读写优盘。

(1) 控制传输与枚举

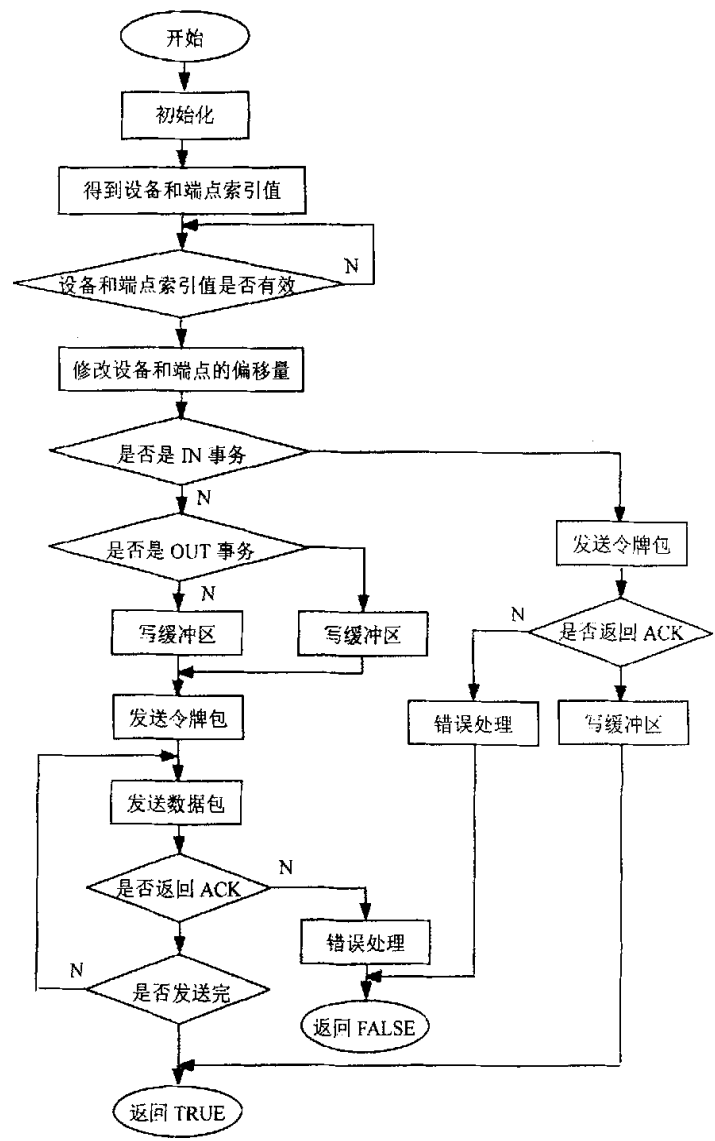


图 4-3 Transaction 流程图

控制传输流程如图 4-4 所示。

对应于 USBControlTransfer()。初始设置事务, 发送数据事务, 获取状态信息事务三个 Transaction 对应一个控制传输中的三个步骤。

设备枚举模块完成对 USB 设备进行检测, 当 SL811HS 检测到有外部 USB 设备接入, 对这个 USB 设备进行复位操作, 检测该 USB 设备是高速还是低速设备, 并设置好 SL811HS 中对应的寄存器位。通过 SL811HS 进行设备枚举。获得设备提供的设备描述符, 设置设备地址、数据描述和配置。设备枚举流程如图 4-5 所示。

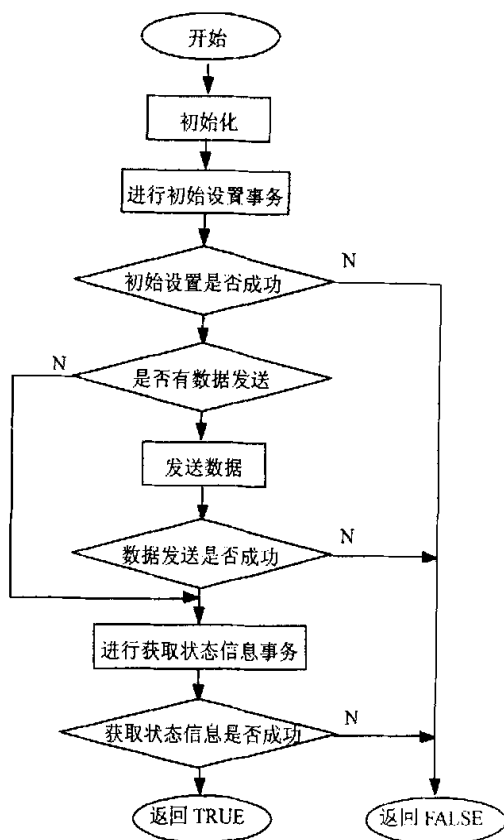


图 4-4 控制传输流程图

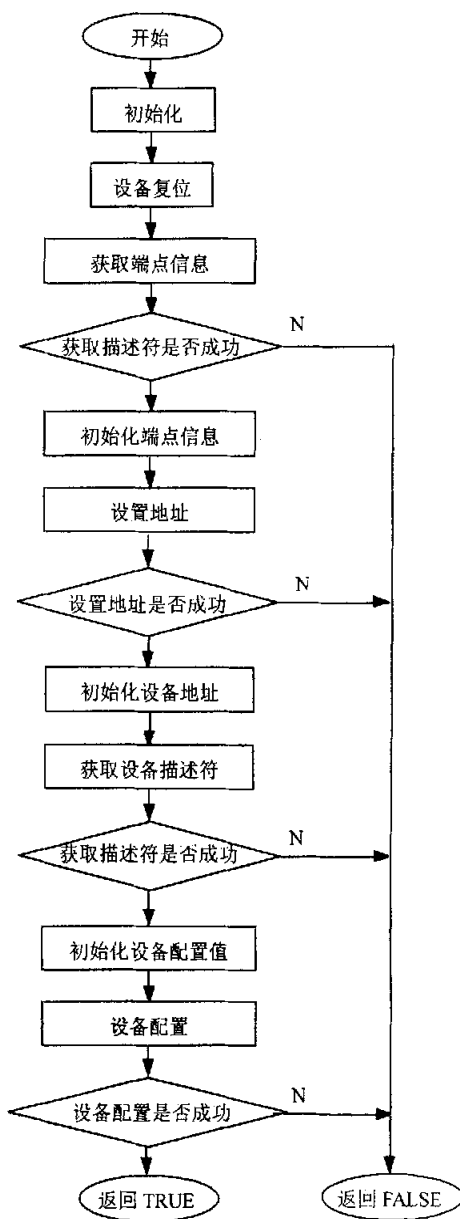


图 4-5 设备枚举流程图

函数 `USBEnumerate()` 将枚举过程分成几个步骤。首先是发送“获取设置描述符”命令，获取端点大小。由 `USBGetDescriptor()` 来实现。得到一定的信息后设置设备的地址，由 `USBSetAddress()` 来完成。然后对设备进行配置，由 `USBSetConfiguration()` 来实现。应该说一个完整的枚举过程不仅仅由这三个步骤组成，还需要进行其他的操作。但是有了这三个步骤基本上就可以进行后面的数据传输了。其中每个步骤其实就是一个控制传输，所以这三个函数都调用了控制传输函数 `USBControlTransfer()`。

`USBControlTransfer()` 要将控制传输分成两个或三个步骤（数据包是可选的如果有数据步骤则为三个步骤），每个步骤都是一个事务，这个事务就由 `USBTransaction()` 来完成。而每个 `USBTransaction()` 都要经历三个阶段（实现三个包的发送）。

枚举的过程可大致用图 4-6 来概括：

下面以获取设备描述符为例来说明控制传输的各个阶段包传输的格式。

设置阶段：首先由主机发出一个 `SETUP` 令牌包，表示将要开始一个控制传输。端点号使用端点 0，然后主机发出一个 `DATA0` 数据包，其中包含着 8B 的“请求设备描述符”命令。如果设备同意该请求，使用 `ACK` 握手包做出应答，否则不予理睬。

数据阶段：设备收到来自设置阶段的命令后，经译码获知为“请求设备描述符”命令，设备便准备向主机发送设备描述符。首先由主机发出 `IN` 令牌包，然后设备向主机发送数据包，数据包中包含着主机所要求的设备描述符。一次传输可能传输不完主机要求的全部数据，可使用多个 `IN` 事务来完成，首先按照最大数据负载传送，最后一次传输剩余的数据。主机每次成功接收到数据包后必须用 `ACK` 来和设备握手。

状态阶段：如果主机所要求的设备描述符传输完毕，主机向设备发出一个长度为 0 的数据包，设备必须对这个数据包做出应答。到此，整个

控制传输结束。

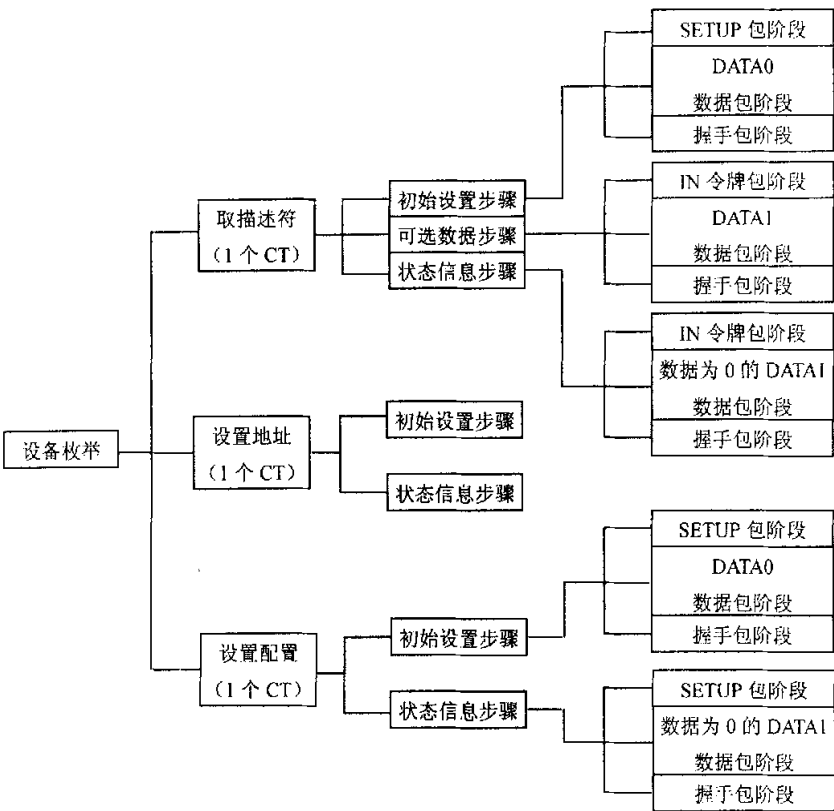


图 4-6 枚举过程

(2) 批量传输

本系统设计时使用到传输协议为 USB Mass Storage Class Bulk-Only Transport 和 USB Mass Storage Class UFI Command Specification。Bulk-only 传输模块按照规范协议将 SCSI-2 指令作成 CBW 命令包，由主机

向设备送出输出请求, 在没有传输失败的情况下, 发出输入请求, 来接收命令请求所要获得的数据, 而后发出输入请求得到 CSW 状态包, 用以判断数据的传输是否正常。

枚举完成以后进行 Bulk-only 传输。批量传输相对要简单一些, 每个 BT 实际就是由一个或若干个类似的 USBTransaction() 来完成。所以首先要判断数据传输的大小, 决定是要由一个 USBTransaction() 来完成还是要多个 USBTransaction() 来完成。那么每个 USBTransaction() 内部要做的事情和 CT 中 USBTransaction() 中要做的事情一样, 因为不管什么传输, 每个事务都要求一样的过程, 也即三个阶段。它们调用的是同一个 USBTransaction() 函数。

(3) 异常处理

USB 传输异常处理是当批量端点出现停止 (STALL) 条件, 即主机发出输入、输出请求时设备端一直出现无响应状态 (NAK), 主机端所采用的主动纠错, 使 USB 接口重新使能。

4.4 Mass Storage 类协议层的实现

当完成设备枚举, 设置设备地址、数据描述和配置后, 通过 Mass Storage 协议访问文件系统。发送读/写命令, 对设备进行读写, 最后反馈响应码。USB 磁盘驱动层实现 Bulk-only 规范传输命令/数据/状态。大容量存储类设备文件系统格式通常为 FAT, 所以主机端文件系统层采用了 FAT32 文件系统。

由 Mass Storage 层、系统软件层就可以实现对 USB 接口的大容量存储类设备的驱动和读写访问。

本系统是实现对优盘的数据传输, 所以只要实现 Mass Storage 类协议, 实现 USB 协议定义的各种标准设备请求。

USB Mass Storage 类是 USB 协议所定义的所有设备类中的一种, 主要用于磁盘管理及大容量数据传输。这一协议框架主要是建立在 19 个 UFI 命令之上的, 实现了这 19 个 UFI 命令, 那么也就建立起了整个协议框架。这 19 个命令主要就是格式化磁盘、读磁盘、写磁盘和读磁盘容量等一系列磁盘管理的内容。

我们设计的系统使用了大容量存储类规范中的 SCSI-2 子类, 所以主机也要实现 SCSI-2 子类功能, SCSI-2 子类下有多个命令, 在这一层就要逐个实现这些命令, 实际就是将这些命令打成标准的 BOT 协议包, 然后调用底层的批量传输。

这一层实现的是 SCSI-2 命令集中的命令。本系统中用到的最多的命令是 INQUIRY、READCAPACITY、READ10。在实现的时候, 每个命令都是一个 BOT 传输, 而一个 BOT 传输又要分成三个批量传输来完成。所以这一层要完成的功能就是将每个命令用一个 BOT 来完成: SCSIInquiry()、SCSIReadCapacity10()、SCSIRead10(), 每个命令调用一个 BOT 传输 MSCBulkOnlyTransport(), 再将一个 BOT 分解成三个批量传输来完成。

图 4-7 所示为 Mass Storage 层结构。INQUIRY、READ CAPACITY、READ(10)等均为一个 BOT。命令传输、数据传输、状态传输均为一个批量传输。

4.5 文件系统层

文件系统将对优盘中的结构进行系统化。有了文件系统, 就可以将优盘中杂乱的数据组织起来, 被主机管理。

使用大部分移动存储类设备符合的 FAT 文件系统。整个文件系统通过 Mass Storage 的 UFI 命令与外设的磁盘建立联系, 并在 USB 主机体系中建立起一个外设磁盘的空间结构, 建立 FAT 表, 并实现查找文件、写文件、读文件等一系列功能。

在实现了 USB Mass Storage Bulk-Only Transport 协议后, 已经可以通过发送相应的 SCSI-2 设备命令块给优盘来获得优盘上的数据了。但是这时得到的数据是按照逻辑块 (Logical Block) 为单位的, 而不是按照我们所熟悉的驱动器、目录、文件为单位的。若要像在 PC 上使用硬盘

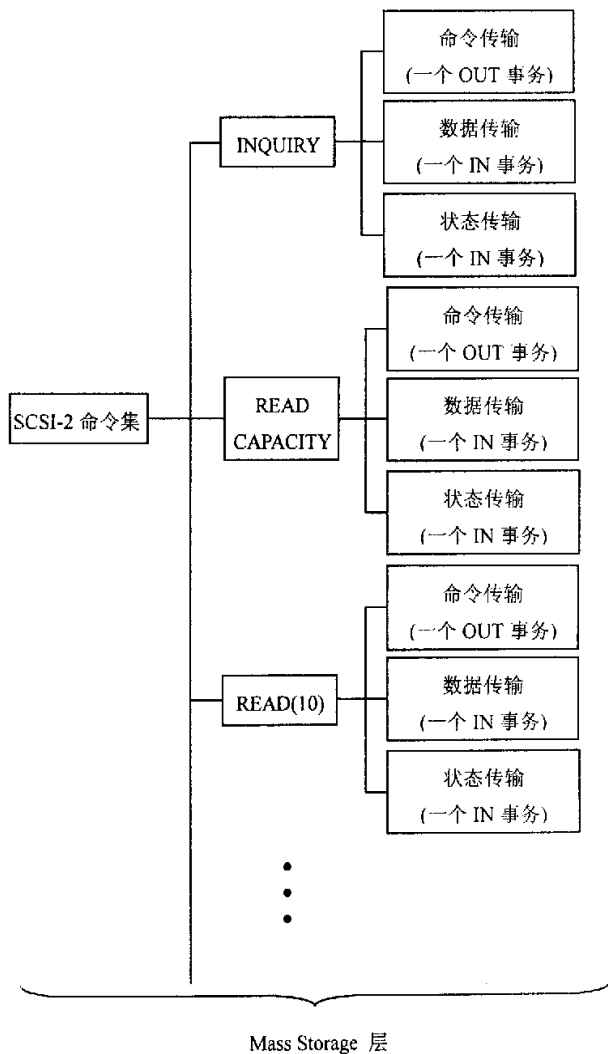


图 4-7 Mass Storage 层结构

那样的使用优盘，在 USB Mass Storage Bulk-Only Transport 协议层之上还需要构建文件系统。如图 4-8 所示，应用程序通过文件系统提供的接口对优盘进行目录、文件以及读写的操作。

文件系统是操作系统中籍以组织、存储和命名文件的结构。当前在 PC 上流行的文件系统主要分为两大类：FAT 和 NTFS。其中 FAT 因为出现时间较早得到广泛的应用，具体又有 FAT12/16/32 之分，目前使用最广泛的是 FAT32。考虑到嵌入式系统资源有限且应用领域相对单一，因此同时实现多套文件系统是没有必要的，为此本系统只设计了一套精简的 FAT32 文件系统。

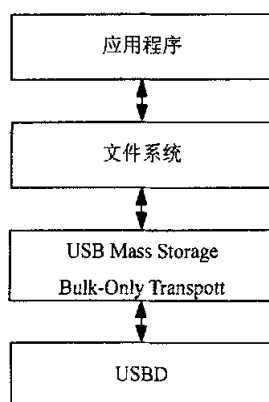


图 4-8 文件系统层在 USB 主机系统软件中的位置

(1) 硬盘逻辑结构

如前所述，优盘在逻辑上具有同硬盘相同的结构，其最小逻辑单位为扇区（Sector），若干个扇区组成一个簇（Cluster），一个文件占用一个或多个簇。下面介绍的有关硬盘的逻辑组织结构也适用于优盘，为此不区分优盘和硬盘。

硬盘的逻辑组织结构如图 4-9 所示, 不难看出目录和文件在硬盘上都是按照文件的形式存放的, 两者的不同之处只是文件属性不同而已。

一块磁盘上可以存在多个逻辑的分区, 每个分区由一定数量的连续的扇区构成, 每个分区的第一个扇区被称为该分区的逻辑零扇区, 它对应着某个物理扇区。一个或多个分区构成一个卷, 文件系统是针对卷而言的。但就 FAT 文件系统而言, 一个卷包含一个分区, 因此分区的逻辑零扇区也就是卷的逻辑零扇区。在每块硬盘的物理零扇区上都存放了该硬盘上的逻辑分区的基本信息, 也称为磁盘分区表 (Disk Partition Table), 它从物理零扇区的 01BEh 开始, 共 64 字节, 存放了四个分区的信息, 每个分区信息占 16 字节, 分区的起始物理扇区号映射为分区的逻辑零扇区, 通过读取分区的逻辑零扇区, 就可以获得该分区所使用的文件系统的相关信息了。

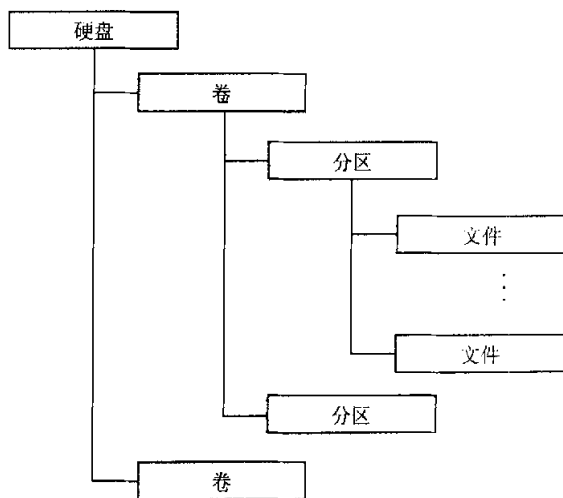


图 4-9 硬盘逻辑组织结构

(2) 精简 FAT32 文件系统

目前的三种 FAT 文件系统：FAT12，FAT16 和 FAT32 的差异就在于文件分配表中每一个条目的大小不同，分别为 12 比特，16 比特和 32 比特。

FAT32 文件系统卷由三个基本部分组成，依次为：保留区（Reserved Region）、FAT 区（FAT Region）和文件与目录数据区（File and Directory Data Region）。如图 4-10 所示，一个 FAT 卷上通常有两个 FAT 区，这两个 FAT 区是完全一致的。因为在 FAT 文件系统中，文件所占用的簇是记录在 FAT 区中的，一旦 FAT 区受损就会导致文件丢失的严重后果，所以在 FAT 区后面还存放了一个 FAT 区的备份，如果 FAT 区的数据受损就利用这个备份来进行恢复。

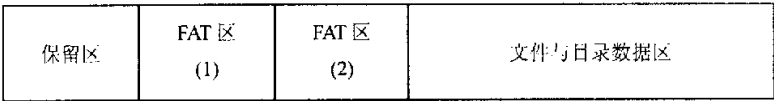


图 4-10 FAT32 文件系统卷结构

1) 保留区

保留区中的第一个扇区是 BIOS 参数块（BIOS Parameter Block，BPB），此扇区有时也称作“引导扇区”、“保留扇区”或者“零扇区”。BPB 含有对文件系统进行识别的关键信息，对于 FAT32 而言，此扇区的结构中并非所有的域在嵌入式系统的文件系统中都有用，这取决于设计者对文件系统的裁减。在设计系统时针对读取优盘数据所需要的域值，保留 BPB_BytsPerSec、BPB_SecPerClus、BPB_RsvdSecCnt、BPB_NumFATs、BPB_TotSec32、BPB_FATSz32 和 BPB_RootClus 这七个域的值。

2) FAT 区

FAT 区是 FAT32 文件系统卷上最重要的一个部分。前面提到，磁盘上

的文件占用一个或多个的簇,也就是说文件在磁盘上占有的最小单位不是扇区而是簇。FAT 区记录了该卷上所有簇的使用情况,根据 FAT 可以找到一个文件占用的所有簇。FAT32 中的每个 FAT 表项对应一个簇,用 32 比特来表示。但是 FAT32 只使用 32 比特中的低 28 比特来表示簇号,若低 28 位全为 0,表示当前 FAT 表项所对应的簇是一个未被占用的“空”簇,可以分配给新建的文件。若低 28 位的值大于或等于簇链结束标志(EOC)的值,则表示该表项所对应的簇是文件占用的最后一个簇;否则,低 28 位表示文件占用的下一个簇的簇号。这样一个文件所占用的簇在 FAT 表中就出一条簇链所对应,只要知道了文件占用的第一个簇的簇号,就可以通过这条簇链找到所有其他的簇。

对给定的簇号 N 可以计算该簇在第一个 FAT 区中所对应的 FAT 表项所在扇区的扇区号:

$$\text{ThisFATSecNum} = \text{BPB_RsvdSecCnt} + (N * 4 / \text{BPB_BytsPerSec});$$

以及该表项在该扇区中的偏移量:

$$\text{ThisFATEntOffset} = (N * 4) \% \text{BPB_BytsPerSec};$$

3) 目录与文件数据区

由于 FAT 系统中 0 号簇和 1 号簇是保留簇,所以目录与文件数据区是从 2 号簇开始的,而 SCSI 命令对磁盘的读写操作都是以物理扇区号为参数的,为此需要计算 2 号簇的第 1 个扇区的扇区号。为简化计算,如无特别说明,下面提到的扇区号均为 FAT 卷的逻辑扇区号,在提供给 SCSI 命令时需要根据磁盘分区表中的数据计算相应的物理扇区号。

对于 FAT32 系统,不存在根目录区的概念,因此根目录所占用的扇区数(RootDirSectors)为 0。可以计算出目录与文件数据区的第一个扇区:

$$\text{FirstDataSector} = \text{BPB_RsvdSecCnt} + (\text{BPB_NumFATs} + \text{BPB_FATSz32});$$

对给定的簇号 N 可以计算出该簇第一个扇区号为

```
FirstSectorofCluster = ( ( N - 2 ) * BPB_SecPerClus ) +  
FirstDataSector;
```

目录与文件数据区中的数据可以分为两大类，一类是文件数据，一类是目录（或文件名）数据。文件数据的结构因文件格式不同而不同。

（3）文件定位算法

前面简要介绍了 FAT32 文件系统规范。在此将介绍如何根据给定的文件路径找到该文件所占用的第一个簇的簇号。得到首簇簇号后可以根据上边给出的方法，计算出该文件所对应的簇链首结点的位置，进而根据簇链访问该文件。

给定文件路径计算首簇的流程如下：

```
begin  
    解析文件路径为 PATH 1/PATH 2/.../PATH n;  
    Header = BPB_RootClus;  
    for ( i = 0; i < n; i ++ )  
    {  
        在以簇 Header 为首结点的簇链中寻找 PATH i 所对应的短目录  
        条目;  
        Header = MAKEDWORD ( DIR_FstClusLO, DIR_FstClusHI );  
    }  
end;
```

（4）嵌入式 USB 主机实现与优盘数据的传输

大规模存储设备驱动是 CPU 和移动存储设备打交道的接口程序。通过调用大规模存储程序，可以得知移动存储设备文件分区表中的 DBR、FAT 表、文件目录表等的起始位置，并可以获得移动存储设备某个地址区间的

一系列数据。嵌入式USB主机不像Windows系统那样可以自动支持大规模存储协议，需要自己编写此协议。

获得FAT32分区表的参数是对FAT32文件系统进行文件访问的基础，本文选择FAT表1起始逻辑扇区号、FAT表的长度、文件数据区起始逻辑扇区号、每簇扇区数、分区总扇区数和根目录的入口首簇号作为一组参数。首先读取主引导扇区到缓冲区并进行结束标志判断；然后根据第一分区入口处读取该分区的分区类型信息，判断是否为FAT32分区格式；再根据第一分区入口处提供的第一分区的起始逻辑扇区地址信息将系统引导扇区读入到缓冲区并进行结束标志判断；最后根据系统引导扇区提供的BPB参数，经过必要的计算，依次获得FAT32文件系统的相关参数。

4.6 USB 主机主流程功能的实现

这里主流程指的是USB主机运行期间所执行所有功能的宏观定义。

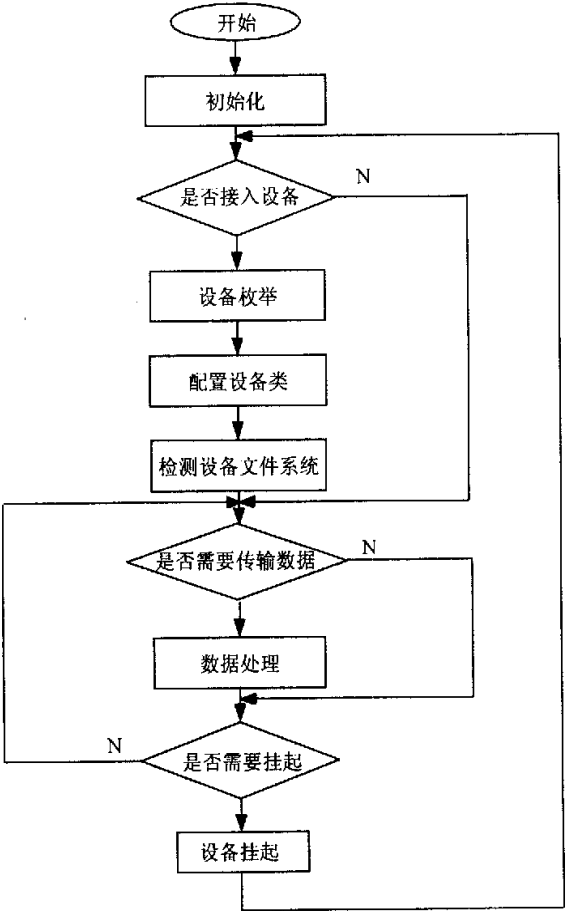


图 4-11 嵌入式 USB 主机软件主流程示意图

第 5 章 系统调试与分析

系统调试采用分层调试的方法，自底向上逐层测试设计的目标。

5.1 硬件驱动层调试

硬件驱动层的调试相对较为简单，是最底层功能的调试。主要完成对 SL811HS 以及 RT12864 数据的读写。也即对以下几个函数进行测试：

```
void SL811BufWrite( int DestAddress,int SourcAddress,int  
Length );
```

```
void SL811BufRead( int SourcAddress,int DestAddress,int  
Length );
```

```
void Write_Command( int Command );
```

```
void Write_Data( int Data );
```

```
int Read_Data( )。
```

可以采用往 SL811HS 以及 RT12864 缓冲区中写入某个值，然后再读取相应的缓冲区中的值进行比较的方法进行测试。

5.2 系统软件调试

HCD 层和 USBD 层是系统软件的核心，该部分的调试也成为系统调试的核心。这里采用自底向上的方法，通过示波器监听 USB 的数据传输过程，将传输过程分成若干个小的部分来调试分析。

USB 中数据的最小单位为域，不同域的不同组合构成不同的包。而数据交换的最小单位为包，在示波器上看到的一帧即为一个包，所以数据包阶段的调试将最终成为系统软件调试的核心。通过示波器上看到的包的格

式来分析所发送数据是否正确以及所收到的数据是否是希望得到的数据。

1. 令牌包

令牌包的格式如图 5-1 所示：

8 位	8 位	7 位	4 位	5 位
SYNC	PID	ADDR	ENDP	CRC5

图 5-1 令牌包的格式

图 5-2 所示是从示波器上捕捉到的一个 IN 包：

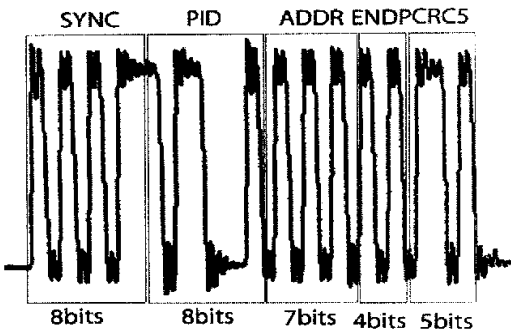


图 5-2 示波器上捕捉到的一个 IN 包

图 5-3 所示是从示波器上捕捉到的一个 OUT 包：

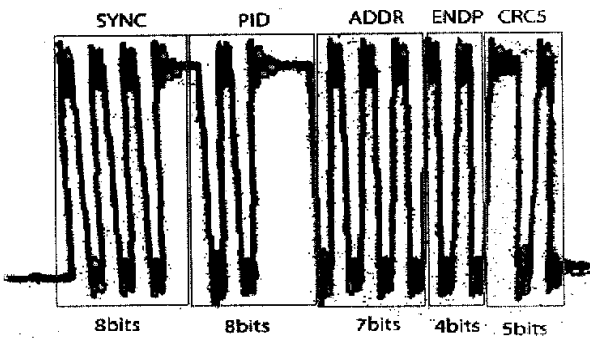


图 5-3 示波器上捕捉到的一个 OUT 包

图 5-4 所示是从示波器上捕捉到的一个 SETUP 包：

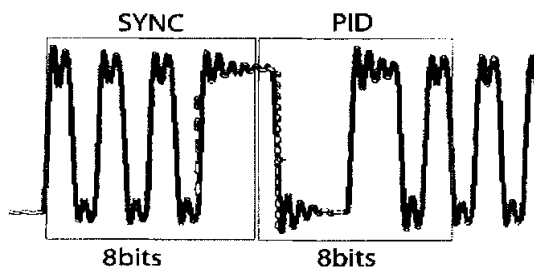


图 5-4 示波器上捕捉到的一个 SETUP 包

帧起始包 SOF 的结构和 IN、OUT、SETUP 包有所不同，其中增加了 11 位的帧起始域 FRAM，代表帧号，而不包括 7 位的地址域和 4 位的端点域。帧起始包不是针对某个特定设备和端点的传输，而是整个 USB 总线动作的一个时间划分。主机要不停地发送 SOF 包来“keep the device alive”。

图 5-5 所示是从示波器上捕捉到的一个 SOF 包：

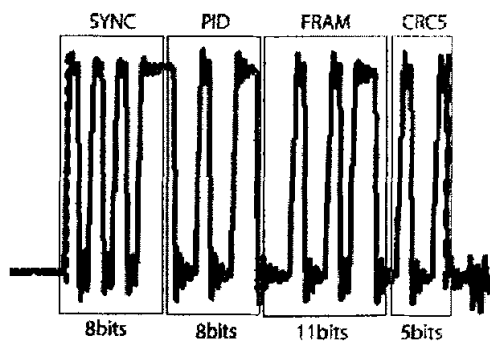


图 5-5 从示波器上捕捉到的一个 SOF 包

图 5-6 所示是等间隔的毫秒帧：

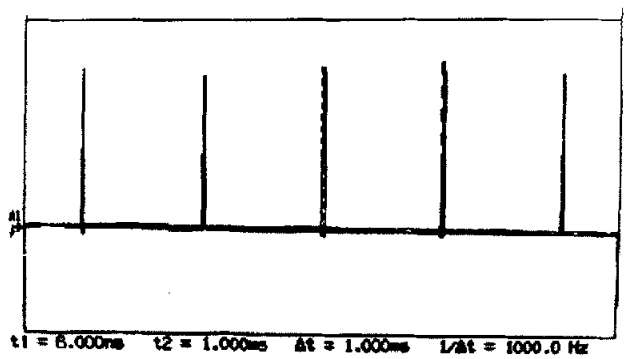


图 5-6 等间隔的毫秒帧

2. 数据包

根据 PID 的不同，数据包（Data Packet）可分为 DATA0 和 DATA1 两种，两种数据包格式都一样，当 USB 发送数据长度超过相应端点容量时必须将数据分成好几个包来发送。如果第一个是 DATA0，则第二个是 DATA1，如此交替进行下去。而接收方也要检查是否是 DATA0、DATA1 交替进行的。令牌包的格式如图 5-7 所示：

8 位	8 位	0~1023B	16 位
SYNC	PID	数据	CRC16

图 5-7 数据包的格式

图 5-8 所示是从示波器上捕捉到的数据长度为 0 的数据包 0：

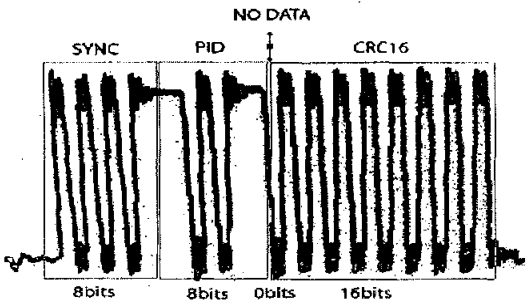


图 5-8 示波器上捕捉到的数据长度为 0 的数据包 0

3. 握手包

握手包（Handshake Packet）结构最为简单，只有 SYNC 域和 PID 域，用于报告数据传输的状态，总共分为三种：确认包 ACK、无效包 NAK、停止包 STALL。ACK 包表示数据正确接收；无效包表示设备无法接受数据或无法发送数据，指设备忙，这种包只能由设备发送；STALL 包也只能由设备发送，表示设备端点被停止或者 USB 设备不支持这个请求。握手包的格式如图 5-9 所示：

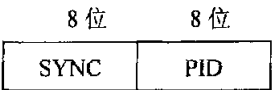


图 5-9 握手包的格式

图 5-10 所示是从示波器上捕捉到的一个被设备 NAK 了的 IN 事务，最后一个包为 NAK 包：

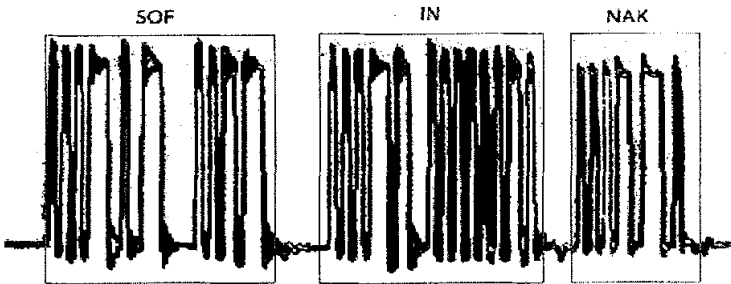


图 5-10 从示波器上捕捉到的一个被设备 NAK 了的 IN 事务

5.3 测试流程

为了测试主机控制器的工作情况，选用一个具体的 USB 设备——朗科的 64M 优盘作为存储媒介进行测试。测试分两步：首先计划完成 SL811HS 和设备的初始化。其次整个测试流程计划完成读取优盘文件并在液晶显示器上显示。

USB 测试流程如图 5-11 所示。

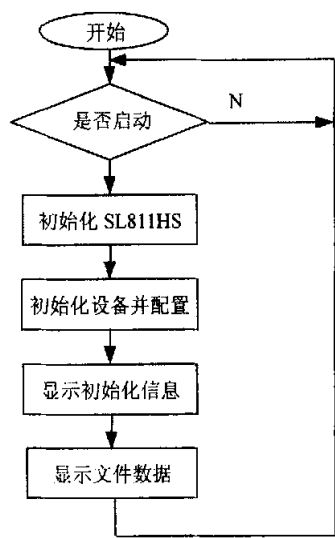


图 5-11 嵌入式 USB 主机测试流程示意图

启动嵌入式 USB 主机系统。在液晶屏上显示如下信息：

HW Reset	硬件复位
OK!	
Initializing	USB 初始化
OK!	
Detecting	检测设备

FS Device!	发现全速设备
Enumerating	枚举
Netac	厂商 ID
Only Disk	产品 ID
1. 11	版本号
010203	文件内容

5.4 应用实例

本系统设计了一个嵌入式 USB 主机，实现与优盘数据传输的功能。针对要实现的功能以及 SPCE061A 强大的语音处理功能的特点，本节以播放存放在优盘上的语音文件的实例来演示系统的总体功能。该语音文件为“.wav”文件，在 PC 机上经过凌阳压缩算法 SACM_A2000 压缩后为“.16k”文件，把此文件存入优盘。

优盘接入本系统后将要播放的语音先从优盘中读出，放入微控制器开辟的区域中，首先擦除以 0XC200H 为首地址的 10K 数据空间，每次擦除 256 字节。非自动方式播放的程序如下：

```
Play_Address = 0xc200 ;  
    SACM_A2000_Initial(0);           //非自动方式播放的初始化  
    SACM_A2000_InitDecoder(1);  
    while(SACM_A2000_TestQueue()!=1)//测试并获取语音队列的状态  
    {  
        Ret=SP_GetResource(Play_Address);  
            //从资源文件里获取一个字型语音数据  
        SACM_A2000_FillQueue(Ret);  
            //获取语音编码数据并填入语音队列等候解码  
        Play_Address++;
```

```
}  
while(1)  
{  
    if(SACM_A2000_TestQueue()!=1)  
    {  
        Ret =SP_GetResource(Play_Address);  
        SACM_A2000_FillQueue(Ret);  
        Play_Address++;  
    }  
    if(Play_Address< 0xfe00 )  
        //如果该段语音未播完，即未到达末地址时  
        SACM_A2000_Decoder();  
    //获取资源并进行解码，再通过中断服务子程序送入 DAC 通道播放  
    else  
    {  
        SACM_A2000_Stop();  
        break;  
    }  
    //否则，停止播放  
}
```

对于 SPCE061A，录放音的编解码是调用库函数完成的，只要完成数据的存入和读出即可。利用本系统可以实现特定人和非特定人语音识别等，将训练的标准样板存于优盘中，训练过程也主要是调用库函数 BSR Train 来完成。

5.5 分析

基于 SPCE061A 的嵌入式 USB 主机的设计, 实现了非 PC 机做为主机与 USB 设备进行数据交换。在本系统的基础之上, 如果扩展 RAM、ROM, 通信模块 (如 RS232、RS485 以及 CAN 总线)、数据采集模块, 采集各种需要测量、控制的信息, 并且最终以文件的形式存储在优盘等 USB 移动数据存储设备中, 为数据的提取、分析提供极大的方便。可以满足更多的 USB 设备类的需要, 可以完成更复杂的功能。

为了在不同的主控制器 CPU 上尽可能方便的实现 USB 主机的功能。在开发嵌入式 USB 主机, 编写底层驱动程序时, 就需要考虑整个系统的可移植性, 使有关 USB 通信的各个任务模块化。

嵌入式 USB 主机作为 USB 的延伸, 国内外起步的时间不长, 不依靠任何操作系统的 USB 主机系统的研究具有一定的领先和前瞻性。

可以预见, 嵌入式 USB 主机最大的市场和增长点在于移动设备, 比如 PDA 与 USB 记忆棒传递数据, 嵌入式监控系统采集数据等, 因此系统的研究具有很强的应用背景和市场前景。

5.6 结论

本文提出了基于 SPCE061A 嵌入式 USB 主机的设计思想, 分析了嵌入式 USB 主机的结构, 通过选择 USB 接口芯片 SL811HS, 液晶显示芯片 RT12864 进行了嵌入式 USB 主机的软硬件的设计。实施了对系统软件的分层处理, 使各层从硬件相关过渡到硬件无关, 为系统的移植奠定了基础。采用了模块化的设计理念, 提供了友好的人机界面。选用 FAT32 文件系统构建了文件系统平台。使系统以文件形式传输数据, 并使其在嵌入式 USB 主机系统中传递的数据也能在 PC 机中识别, 增强了存储数据的兼容性。通过与 USB Mass Storage 类设备进行数据传输的测试。实现了 Mass Storage

类协议框架。经实验验证，本系统可以实现与 USB 设备进行包括语音数据在内的数据传输和交换。达到了预期的设计要求，为移动存储设备拓展了应用空间。

研究生期间发表的论文

- [1] 张燕宁. 基于 MAX125 的多通道同时采样数据采集系统. 电力建设. 2003. 11
- [2] 张燕宁. 黄伟华. 侯建军. 采用一种新的马尔可夫模型进行癌症医学图像分割. 北方交通大学学报. 2003. 5
- [3] 杨洋. 张燕宁. 吴剑飞. 基于 MPC850 实现 ADSL 宽带网接入系统. 北方交通大学学报. 2003. 6

参考文献

- [1] 陈启美,丁传锁.编著.计算机USB接口技术.南京大学出版社 2003.1
- [2] [美]Jan Axelson 著.陈逸 等译. USB 大全中国电力出版社 2001.5
- [3] 马伟.编著.计算机USB系统原理及其主/从机设计.北京航空航天大学出版社.2004.1
- [4] 肖踞雄,翁铁成,宋中庆.编著.USB 技术及其应用.清华大学出版社 2003.1
- [5] MindShare,Inc.Don Anderson.Dava Dzatk 著.孟文译.USB 系统体系(第二版)中国电力出版社.2003.9
- [6] 李肇庆,廖峰,刘建存.编著.USB 接口技术.国防工业出版社.2004.6
- [7] 张弘 编著.USB 接口设计.西安电子科技大学出版社.2002.12
- [8] USB Specification Revision 1.1.1998
- [9] Universal Serial Bus Mass Storage Class Specification Overview Revision 1.1.2000
- [10]李维,郭强.编著.液晶显示应用技术.电力工业出版社 .2000.3
- [11] 李朝青. 编著.PC 机及单片机数据通信技术. 北京航空航天大学出版社 2000.12
- [12] 李品皎.编著.嵌入式语音技术及凌阳 16 位单片机应用.北京航空航天大学出版社 2003.11
- [13] 罗亚飞.等编著.凌阳 16 位单片机应用基础.北京航空航天大学出版社 2003.12
- [14] Cypress Semiconductor Corporation SL811HS/SL811HOST Application Notes.
- [15] Cypress Semiconductor Corporation.SL811HS Embedded USB Host/Slave Controller.Revised March 14.2002

- [16] 朱勇, 陈艳, 虞鹤松. 数字机顶盒中 USB 大容量存储类主机的设计实现. 电视技术. 2004. 1
- [17] 李洪奎, 孙德田, 盛岩. 通用串行总线及其芯片应用. 信息技术 2002. 8
- [18] 姜波, 陈英, 邹静, 胡涛. 基于 OHC 的 USB 系统底层接口的实现. 电子技术. 2003. 1
- [19] 邵同震, 孙兴民. 嵌入式系统中 USB 总线驱动的开发与应用. 计算机应用研究. 2002. 12
- [20] 梁宏军, 马鸣锦. 通用串行总线数据传输模型. 现代电子技术. 2003. 10
- [21] 罗悦悻, 沈建华. 嵌入系统的硬盘文件操作. 计算机工程. 2004. 2
- [22] 孙庚, 蒋国强. USB 块传输设备的设计与实现. 计算机工程 2003. 6
- [23] 杨孝宗, 崔刚. USB HOST & HOST Controller 剖析与实现赵蕴龙. 小型微型计算机系统. 2003. 1

致 谢

首先深深的感谢我的导师侯建军教授，他在我攻读硕士学位期间，从课程选择、论文发表、学位论文选题、研究工作及学位论文的撰写过程都给予了悉心的指导和鼎力相助。创造了良好的学术氛围，使我得以提高学术水平。

感谢学院的领导在整个攻读硕士学位期间给予了我极大的关怀和支持，为我提供了精神上的支撑。感谢周围的同事们对我伸出了一双双友情的手，帮助我坚持完成学业。感谢众多的同学们给了我极大的帮助。

向所有支持、关心、帮助我的老师、同事、家人、朋友和同学们表示深深的谢意。